



Enhancing Network Failure Mitigation with Performance-Aware Ranking

Pooria Namyar and Arvin Ghavidel, *University of Southern California*;
Daniel Crankshaw, Daniel S. Berger, Kevin Hsieh, and Srikanth Kandula,
Microsoft; Ramesh Govindan, *University of Southern California*;
Behnaz Arzani, *Microsoft*

<https://www.usenix.org/conference/nsdi25/presentation/namyar>

This paper is included in the
Proceedings of the 22nd USENIX Symposium on
Networked Systems Design and Implementation.

April 28–30, 2025 • Philadelphia, PA, USA

978-1-939133-46-5

Open access to the Proceedings of the
22nd USENIX Symposium on Networked
Systems Design and Implementation
is sponsored by



Enhancing Network Failure Mitigation with Performance-Aware Ranking

Pooria Namyar[‡], Arvin Ghavidel[‡], Daniel Crankshaw^{†*}, Daniel S. Berger[†],
Kevin Hsieh[†], Srikanth Kandula[†], Ramesh Govindan[‡], Behnaz Arzani[†]

[‡]University of Southern California, [†]Microsoft

Abstract— Cloud providers install mitigations to reduce the impact of network failures within their datacenters. Existing network mitigation systems rely on simple local criteria or global proxy metrics to determine the best action. In this paper, we show that we can support a broader range of actions and select more effective mitigations by directly optimizing end-to-end flow-level metrics and analyzing actions holistically. To achieve this, we develop novel techniques to quickly estimate the impact of different mitigations and rank them with high fidelity. Our results on incidents from a large cloud provider show orders of magnitude improvements in flow completion time and throughput. We also show our approach scales to large datacenters.

1 Introduction

Datacenter networks often incur a variety of (concurrent) failures ranging from failed or lossy links to localized, persistent congestion. Repairing these failures takes time [1]. For example, operators require days to replace optical links and hours to fix hardware-induced packet corruptions [11, 21, 63, 71]. As a result, cloud providers install mitigations to reduce the impact of failures while they work on repairs. In this paper, we focus on *network-level* mitigations¹ such as disabling links or switches and re-routing traffic. These are effective because of the path and resource diversity in datacenter networks.

Given the economic importance of cloud services and the increasing likelihood of failures at scale, it is essential to find and implement effective mitigations quickly. Today, cloud providers are increasingly using automation to mitigate each incident. For example, Azure uses automation for nearly 80% of its incidents. An auto-mitigation system allows an operator to pre-define a limited set of potential mitigations for each type of failure. The system then chooses the “best” mitigation for each individual incident. At its core, an auto-mitigation system *ranks mitigations* based on one or more criteria, and it must do so *quickly* to be effective. At Azure, mitigations must be in place within 5 minutes [21] of failure localization.

Azure uses *local* criteria to assess mitigations.² For example, disabling a link is acceptable if it leaves sufficient functional uplinks at the corresponding switch. The state of

Approach	Metric	E	G	U	B	S	P
NetPilot	Util/Drop	×	✓	×	✓	✓	×
CorrOpt	#Paths	✓	✓	×	×	✓	×
Operator	#Uplinks	×	×	×	✓	✓	×
SWARM	FCT/Tput	✓	✓	✓	✓	✓	✓

TABLE 1: SWARM is the only method that mitigates failures based on **E**nd-to-end **G**lobal **P**erformance metrics, considers **U**ncertainty in future networking behaviors, supports a **B**road range of actions and failure, and **S**cales to large datacenters. (**E**: End-to-End, **G**: Global, **U**: Uncertainty, **B**: Broadly applicable, **S**: Scalable, **P**: based on Performance)

the art either uses *global proxy* metrics, such as the residual path diversity from the top-of-rack (ToR) switches to the spine of the datacenter [71], or *global non-end-to-end* measures like packet loss and network utilization [63]. However, these methods can negatively impact customers by suggesting inadequate mitigations (§2).

In this paper, we explore a new mitigation ranking criterion (Table 1): the impact on the *end-to-end* connection-level performance (CLP) metrics, throughput and flow completion time. We can quantify the *global* impact using distributional measures of these quantities (averages and percentiles) across all connections in the datacenter. Failures adversely impact these global end-to-end metrics, and an ideal mitigation is the one that minimizes the impact. For instance, if our goal is to optimize 1st percentile (1p) throughput, the best mitigation is the one with the least impact on 1p throughput (§3).

From a cloud operator’s perspective, this criterion reflects the network performance that customers experience and is preferable to local or non-end-to-end metrics that may not correlate with customer-visible network performance. However, at the scale of modern datacenters, the feasibility of quickly ranking mitigations based on global end-to-end CLP measures is unclear.

We introduce SWARM, a service for operators and auto-mitigation systems that quickly ranks mitigations while scaling to large clusters. SWARM leverages the insight that ranking mitigations only require an estimate of CLP distributions to produce an effective *ordering*. Its *CLP estimator* models traffic, routing, and transport behavior in sufficient detail to ensure ranking fidelity and, at the same time, produces results in just a matter of minutes.

To approximate CLP, SWARM must estimate *per-flow* performance not just for the current network state (topology and

*The author contributed to this work while at Microsoft.

¹ We use network-level mitigations and mitigations interchangeably. We briefly discuss application-level mitigations in §3.4.

² Its auto-mitigation system uses local criteria to determine whether taking a fixed action is better than taking no action.

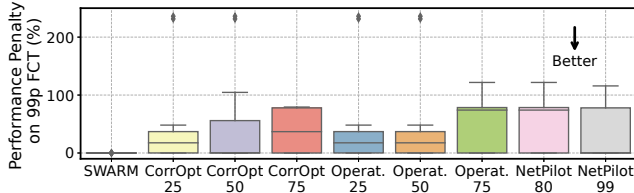


FIGURE 1: Our solution (SWARM) takes orders of magnitude better decisions by ranking mitigations using approximates of flow-level metrics and by accounting for uncertainties. The results are from scenario 1 in Mininet, see §4.2 (Operat. = Operator).

routing) but for potential (unknown) future network states that may manifest while mitigation is in place. Flow performance can depend on other concurrent flows as well as flow arrivals and departures. The central challenge in SWARM is producing an estimate quickly while accounting for all these factors.

SWARM overcomes these challenges as follows (§3):

- It takes flow arrivals, sizes, and communication probabilities as input distributions. It then samples a set of flow-level traffic traces to ensure statistical significance. For each demand, it generates routing samples to capture uncertainty in flows' paths. SWARM estimates CLP for each sample and then combines these estimates to create a composite distribution that succinctly captures traffic and routing variability. It uses this distribution to rank mitigations.
- SWARM estimates CLP separately for long and short flows. Estimating CLP for short flows is easier since they experience less time-varying network behavior. SWARM takes care in modeling long flows along two dimensions: (1) whether their throughput is limited by loss or contention, and (2) how this limitation changes over time as flows arrive/depart and network bottlenecks shift.
- It uses a suite of aggressive scaling methods, which include pipelining, parallelism, topology downscaling, and careful data structure design to estimate CLP quickly.

Using CLP estimates allows SWARM to explicitly account for failure characteristics (*e.g.*, packet drop rate), reason about a broader range of mitigations (taking no action or bringing back a previously disabled link), and model failures (*e.g.*, packet drops below the ToR) that previous methods [63, 71] cannot (see Table 1).

In summary, we make the following contributions:

- We propose CLP-aware failure mitigation, which finds the mitigation with the least impact on network performance. This is a significant departure from state-of-the-art.
- We identify sufficient approximations that allow us to build a robust and scalable CLP estimator that helps rank mitigations effectively (Fig. 1) and support a broader range of failures and mitigations compared to prior work (Table 1).
- We show SWARM is fast at scale and useful. For common failure scenarios (Scenarios 1 and 2 in §4), it picks either the best mitigation or one that is at most 9% worse than the best mitigation. It also outperforms the state-of-the-art by orders

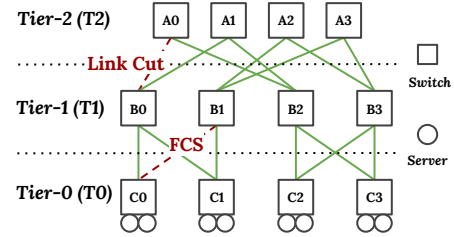


FIGURE 2: An example of two consecutive failures. First, the link between C0 and B1 experiences frame check sequence (FCS) errors. After mitigating, but before fixing this failure, a fiber cut between A0 and B0 causes congestion-induced packet drops.

of magnitude. In more complicated failure cases which many prior work [63, 71] do not support, SWARM picks an action that is only 29% worse than the best mitigation.

2 CLP-Aware Mitigation

We use simplified versions of real-world incidents at Azure (Fig. 2) to explain the limitations of state-of-art auto-mitigation techniques and to illustrate the benefit of CLP-based impact assessments.

Failure scenario. Multiple link failures are common in cloud providers [7, 31, 55]. We emulate this in the Clos topology in Fig. 2. First, frame check sequence (FCS) errors [71] appear on a link. Operators mitigate this failure, but before they can physically replace the link, a fiber cut on another link (LINK CUT) causes congestion and packet loss.

We emulate (in Mininet [37], details in §4) a sequence of flow arrivals and successively apply each mitigation (or a combination of them) for FCS and LINK CUT. We use HIGH FCS and LOW FCS to denote the drop rate of $\sim 5\%$ and 0.005% respectively. For this example, our goal is to maximize the 1st percentile (1p) throughput³. We show how using Azure's troubleshooting guide, CorrOpt [71], and NetPilot [63] lead to substantial and unnecessary performance degradation.

Troubleshooting guides in Azure disable any failed link (with drop rate $\geq 10^{-6}$) if at least half of the switch uplinks are healthy. This mitigation for FCS achieves a 1p throughput of 3.6 Mbps and is optimal when the drop rates are high (HIGH FCS). However, it is conservative and static, and ignores the failure pattern, link location, and traffic demand. For example, leaving the lossy link in place in LOW FCS and taking no action has a higher 1p throughput (14.2 Mbps), while disabling the link causes congestion and impacts tail performance. For LINK CUT, Azure's guidelines do not do anything in the face of congestion, which results in a 1p throughput of 2.7 Mbps. In this case, the 1p throughput is higher if we adjust WCMP [70] weights to reduce traffic on congested links (3.2 Mbps).

CorrOpt [71] disables the link in FCS if there is sufficient path diversity. This is sub-optimal for the same reason dis-

³SWARM can optimize quantiles of both throughput and FCT.

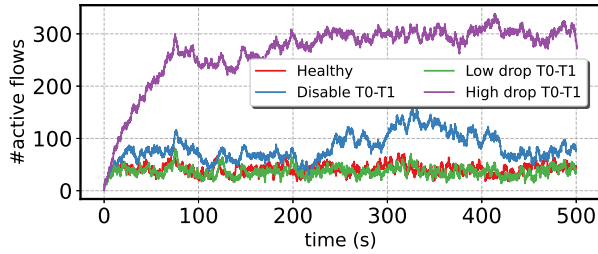


FIGURE 3: Failures and mitigations can increase flow durations, resulting in more active flows. (Fig. 2 topology in Mininet)

cussed above: depending on the failure properties (*e.g.*, drop rate and location), taking no action may lead to a higher 1p throughput. CorrOpt focuses on FCS errors and does not consider congestion induced by capacity drops, like LINK CUT.

NetPilot [63] always disables the faulty link to optimize one of its health metrics (loss rate). This may not be the best option. In a LOW FCS scenario, disabling the link drops the 1p throughput from 15 Mbps to 3.6 Mbps. After the LINK CUT, NetPilot disables the congested link or switch to avoid additional drops. This exacerbates the problem and reduces the 1p throughput to 3.17 Mbps. A better strategy is to undo the previous mitigation and re-enable the LOW FCS link when LINK CUT occurs, which results in 1p throughput of 14.2 Mbps.

Takeaways. While this is a simplified example, operators can negatively impact customers in practice and cause extended outages if they fail to find an effective mitigation [2]. Rules with static thresholds in troubleshooting guides cannot capture correct mitigations because CLP impact depends on traffic demands. Path diversity measures (as in CorrOpt) cannot capture customer impact since they do not account for the failure characteristics. Non-end-to-end metrics like packet loss or utilization (as in NetPilot) often suggest disabling links, which discounts better mitigations. In contrast, SWARM ranks mitigations based on impact on end-to-end global measures.

3 SWARM Design

Network operators seek to improve flow completion time (FCT) and throughput in their datacenters [5, 6, 41, 44, 61]. These CLP metrics evaluate the performance properties of importance for short and long flows. Moreover, operators express their objectives as distributional statistics over the entire datacenter (*e.g.*, tail or average performance). In this section, we describe how SWARM ranks mitigations in terms of their impact on CLP metrics, throughput and FCT.

3.1 Challenges and Insights

Challenges. To optimize CLP objectives, SWARM must quickly estimate the distributions with sufficient accuracy to ensure effective mitigation ranking. This is hard:

Traffic characterization. SWARM requires information about traffic demands to estimate CLP distributions for a given failure and mitigation. Instantaneous flow or ToR-level

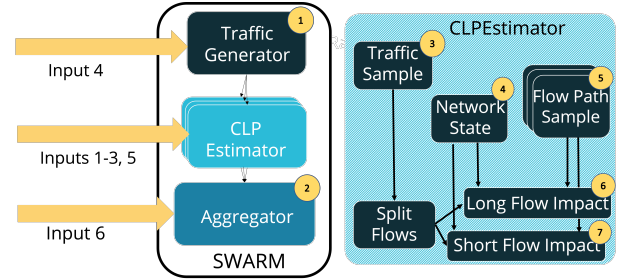


FIGURE 4: SWARM Design.

traffic matrices (TMs) can provide this information. However, fine-grained flow-level TMs are impractical to capture at datacenter scales and are sensitive to failures and mitigations. For example, packet drops often extend the flow durations, resulting in 3 - 4 $\times$ more concurrently active flows (Fig. 3). ToR-to-ToR TMs, as in NetPilot [63], are too ambiguous since they aggregate flows with different characteristics (*e.g.*, capacity drops impact long flows more than short flows).

Routing determines the contention level at each link, which impacts throughput and FCTs. It depends on ECMP hash functions, as well as any existing failures in the network. The ECMP hash functions can change when links fail or switches reboot [56]. SWARM must consider these factors.

Transport behavior. SWARM needs to model transport behavior since CLP measures depend on congestion control algorithms (*e.g.*, BBR vs. Cubic), their parameters (*e.g.*, initial window size), and their reaction to failures (*e.g.*, packet drops). However, it is hard to model transport behavior while maintaining scalability. Accurate simulations over-index on specific protocols and are slow [29]. Faster approximate simulators do not account for lossy links [67, 68] or require a prohibitive amount of compute [67]. Existing formal models have limitations (see §6). For instance, fluid models [36] capture steady-state for long flows, but datacenter flows are often short and do not reach a steady state [44].

Temporal and spatial dependencies. CLP measures depend on the time-varying number of flows competing for bandwidth at a link. They also depend on where and for how long these flows experience congestion. For example, a flow bottlenecked at a link does not need its full fair share at other links. These bottlenecks may shift frequently due to flow arrivals and departures, and SWARM must capture them.

Approach. At the core of SWARM (Fig. 4) is a *CLPEstimator* that estimates the *distribution* of throughput and flow completion time (FCT) for a given network, failure pattern, and mitigation set.

SWARM avoids the limitations of fine-grained flow-level TMs and uses an *approximate* TM distribution (§3.2). It generates multiple (1) TM samples (3) from three inputs that cloud providers like Azure already collect: the flow arrival rates, the flow size distributions [44], and the probability of server-to-server communication [9].

Failure	Mitigation	Works that consider these failures/actions
Packet drop above the ToR	Take down the switch or link	NetPilot, CorrOpt, Operators
	Bringing back less faulty links to add capacity	×
	Changing WCMP weights	×
	Do not apply any mitigation	×
Packet drop at ToR	Disable the ToR	Operators
	Move traffic <i>e.g.</i> , by changing VM placement	×
	Do not apply any mitigation	×
Congestion above the ToR	Disable the link.	NetPilot, Operators
	Disable the device	NetPilot, Operators
	Bring back less faulty links to add capacity	×
	Change WCMP weights	×
	Do not apply any mitigation	×

TABLE 2: List of failures and mitigations in SWARM. Disabling a device is a common mitigation for congestion [63], see §E.

SWARM addresses routing uncertainty by evaluating CLPs on enough flow path samples (5) to reach a target statistical confidence (§3.3). It generates these samples based on the network state (4).

The trickiest challenge is to model the impact of losses and dependencies between concurrent flows on throughput and FCT. To this end, SWARM uses three techniques.

Epoch-based flow rate estimator. SWARM uses a fast, scalable epoch-based flow rate estimator to handle temporal bandwidth changes and flow dependencies. It divides time into multiple epochs, recomputes CLPs in each epoch, and *combines* the results to find an overall estimate.

Traffic Classification. SWARM needs to quickly find effective mitigations at scale rather than accurately estimating the flow performance, which can be slow [29]. Therefore, SWARM divides traffic into long (6) and short flows (7) based on their sizes [5, 44]. In each epoch, it estimates throughput and flow completion time separately. This approach results in significantly better mitigations (§4) for three reasons:

First, failures affect short and long flow differently. Longer flows are exposed to network variations, while shorter flows only experience a snapshot of the network and have more predictable FCTs.

Second, congestion control algorithms [12, 28] typically have a start-up phase to find available bandwidth. Short flows with few packets may finish during this phase. Therefore, they experience queueing delays caused by switch buffer occupancies rather than bandwidth limits [5, 44]. Accounting for these factors improves the effectiveness of our approach.

Third, previous studies [5, 6, 44] have shown that datacenter traffic is a mixture of short and long flows. Short flows are delay-sensitive, while long flows are throughput-sensitive. SWARM reports these separately and allows operators to adjust their mitigations based on their requirements. For example, they can prioritize short flows if their workload mainly consists of latency-sensitive short flows. Modeling these two classes separately also helps with scalability. Future work can

explore modeling more than two classes of flows, such as those that are not short enough to ignore startup behavior.

Transport protocol abstraction. SWARM uses an approximate model of transport protocols that is effective and scalable: (1) it assumes long flows are TCP-friendly [4, 20], which means each long flow grabs a fair share of the bottleneck bandwidth in the absence of failures. (2) under failures and packet drops, SWARM determines if long flows are *capacity-* or *loss-limited*. For capacity-limited flows, it computes their fair share of bandwidth. For loss-limited flows, it estimates the bandwidth at which the control loop converges under loss. SWARM extends existing max-min fair algorithms [34, 45] to detect and estimate both simultaneously. (3) SWARM assumes short flows do not reach steady-state and are impacted by packet drops or queueing delays rather than the bandwidth limits.

SWARM aggregates distributions (2) from each traffic and routing sample. Operators use these to rank mitigations and set priorities based on one or more distributional metrics (*e.g.*, prioritize average throughput over FCT).

3.2 SWARM: Inputs and Outputs

Inputs. Operators or auto-mitigation tools can invoke SWARM with the following inputs:

1. Datacenter topology.
2. List of ongoing mitigations (if any).
3. Failure pattern (*e.g.*, *estimated* loss rate) and location.
4. Data center traffic details (*e.g.*, TMs distributions).
5. Candidate mitigations to evaluate.
6. A *comparator* that ranks mitigations by CLP estimates.

Inputs 1-2-3. Cloud providers use monitoring systems [13, 40] and automated watchdogs [21] to detect incidents and use different techniques [8, 63, 66] to localize the failure. They then create incident reports [1, 2, 21] that contain details of the incident. On-call engineers or automation systems use these reports to install mitigations that are active until the operator finds the root cause and repairs the failure. These reports contain the information SWARM needs, such as the failure

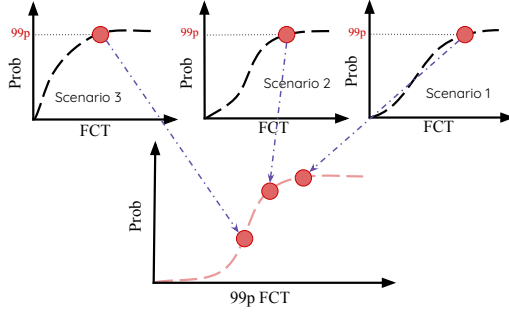


FIGURE 5: The composite distribution of 99p FCT obtained from the FCT distributions of different traffic and routing samples.

location or pattern. The failure characterization (*e.g.*, packet drop rate) is sometimes imperfect, and operators may not be able to accurately localize the failure. SWARM can tolerate errors in packet drop rate (§4), and operators can incorporate the probability of different locations (§5) and iteratively refine mitigations to deal with imperfect localization.

Input 4. SWARM requires simple characterizations of data-center traffic: the flow arrival distribution, the server-to-server communication probability, and the flow size distributions. From these, it extracts a set of flow-level demand matrices (§3.3). These probabilistic characterizations of the inputs allows SWARM to be robust to traffic variability and ensure a desired level of statistical confidence in its estimation.

Input 5. SWARM requires a mapping from the failure types to a list of mitigations or a combination of mitigations. Cloud providers [33, 57] already document possible mitigations for each type of incident in their troubleshooting guidelines and can use these documents to create the mapping. Certain actions may require additional information, such as VM placements or WCMP weights. We assume operators use existing techniques [26, 60] to find values for these inputs. Table 2 shows a sample of failures and associated mitigations.

Input 6. Operators can customize the *comparator*. We currently support two types of comparators, and we can easily extend to others (see §4). The *priority* comparator considers throughput-based and FCT-based metrics in a pre-specified priority order. The *linear* comparator is a linear combination of two or more of these metrics, where the operator specifies the weights. This flexibility enables operators to adjust their mitigation strategy based on their workloads. For example, if delay-sensitive short flows [5, 44] are the dominant workload, they can choose to prioritize the impact on short flows.

Outputs. SWARM outputs the mitigation (or mitigation combination) with minimal impact as ranked by the comparator.

3.3 SWARM: Internals

The CLPEstimator (Fig. 4) takes a demand matrix $\mathcal{T}$ and a mitigation $\mathcal{M}$ as inputs and estimates: (a) the distribution of average throughput across all the long flows in $\mathcal{T}$ and (b) the distribution of FCT across all the short flows in $\mathcal{T}$. SWARM can compute average throughput from FCT and vice

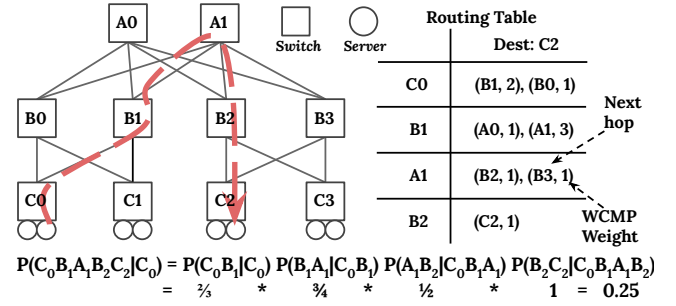


FIGURE 6: The probability of a flow taking a particular path.

versa using $FCT = \frac{\text{flow size}}{\text{throughput}}$ if necessary. SWARM calls the CLPEstimator for each candidate mitigation and ranks these mitigations based on the estimates (see Alg. A.1).

How SWARM uses CLPEstimator. SWARM samples K different demand matrices (① in Fig. 4) and invokes the CLPEstimator to evaluate a given mitigation on each of them. The CLPEstimator then internally generates N different *routing samples* (⑤), where each sample describes the path for every flow in the demand matrix. We choose K and N to ensure a desired confidence level (see below).

Depending on the comparator, SWARM estimates the *distribution* of the *percentiles* of throughput and FCT across the traffic and routing samples (Fig. 5). For instance, if the comparator uses 99p FCT, SWARM would extract the 99p from FCT distributions of the $N \times K$ samples and form a *composite distribution* of the 99p FCTs. The variance of this distribution captures the uncertainty in our estimates, which we can reduce by increasing the number of samples [14] (Fig. A.4).

SWARM uses the composite distribution (②) to compare and rank mitigations. This approach allows it to handle uncertainty explicitly and provide robust mitigation rankings.

Network state representation. SWARM models the network state (④) using a graph $G = (\mathcal{V}, \mathcal{E})$, where each edge e has a capacity and a drop rate (0% = healthy and 100% = down), each node v has a drop rate and a routing table, and each server s maps to a switch. Before each invocation, SWARM *updates* this state to reflect the mitigation (line 2 in Alg. A.1). It uses data structures to ensure this step is efficient (§3.4).

Modeling traffic variability. The demand matrix $\mathcal{T}$ includes the arrival times, flow sizes, and their corresponding source and destinations. To create $\mathcal{T}$, SWARM uses input 4 (①): for each flow, it randomly samples the arrival time from the flow arrival distribution, the source-destination pair from the server-to-server communication probabilities, and the flow size from the size distribution. SWARM then invokes CLPEstimator with K demand matrix samples. It uses the Dvoretzky–Kiefer–Wolfowitz (DKW) inequality [18] to determine K based on a given confidence level α . DKW provides confidence for the difference between an empirically sampled distribution and the ground truth distribution.

Modeling routing uncertainty. The CLPEstimator handles

routing uncertainty by generating N different flow path samples (5), each representing a different routing of the flows (§3.1). First, it uses the DKW inequality [18] to determine the N to achieve a confidence level of α . Then, it samples from the distribution of possible paths between source-destination pairs. It computes the probability of a specific path for a given source-destination pair based on routing tables and associated WCMP weights at each node (Fig. 6).

Modeling the throughput of long flows. SWARM models long and short flows separately (Line 6 and Line 7 in Alg. A.1). Long flows typically reach their steady state, and their throughput depends on network variations and packet drops.

Varying network conditions. SWARM approximates network variations and the arrival and departure of flows that compete on a link by dividing time into discrete epochs (Alg. 1). It assumes stable conditions within each epoch (no flow arrival/departure) but allows for variations across them. At the beginning of each epoch, SWARM adds the newly arrived flows to the set of active flows (Line 6). Then, it computes their bandwidth share (Line 7). At the end of each epoch, SWARM updates the number of transmitted bytes for each flow, removes the completed flows, and records their overall throughput estimates (Line 8-Line 16).

Computing bandwidth share. SWARM finds the rate of each flow within an epoch in two steps (Line 7 in Alg. 1):

1. It computes the loss-limited throughput of each flow, as described below.
2. It executes a demand-aware extension of the water-filling approach [34], which uses the loss-limited throughput estimates as an upper bound on the flow throughput and provably converges in $O(\text{number of edges})$. We have developed this extension (see §A.2).

Modeling loss-limited throughputs. It is possible to compute the throughput under packet drop analytically, but these models are specific to certain congestion control protocols and cannot easily extend to other variants (e.g., [5]). SWARM overcomes this limitation by using an empirically-driven distribution of the loss-limited throughputs. To find this distribution, SWARM measures the average throughput of a long flow under different network conditions (e.g., drop rate, latency) through experiments in a small testbed. In each experiment, it ensures that link capacities are high enough so that they never become bottlenecks, and the drop rate is the only limiting factor. SWARM repeats the experiment multiple times for each network condition to create a robust distribution [18]. Depending on the uncertainty in transport protocols running in the data center, we can also change the mix of transport protocols we use. SWARM then uses this distribution to sample the drop-limited throughputs. See §B for details.

Modeling the FCT of short flows. Prior work [42] develops an analytical model for the average FCT across flows. We are unaware of any models that can estimate the distribution. In developing SWARM, we observe that short flow are more

Algorithm 1: Impact on Long Flows.

Input: G . current network state.
Input: $\mathcal{T} = \{\langle \text{source, destination, size, start time} \rangle\}$.
Input: $\mathcal{R}$. the sampled routing of each flow.
Input: $\mathcal{I}$. measurement interval.
Input: ζ . epoch size.
Output: β_l . distribution of throughput of long flows

```

1  $\mathcal{K} \leftarrow \{\}$ 
2  $time \leftarrow 0$ 
3  $\beta_l \leftarrow []$ 
4 while  $\exists f \in \mathcal{T} : f.start \geq time$  or  $\mathcal{K} \neq \emptyset$  do
5    $time \leftarrow time + \zeta$ 
6    $\mathcal{K}.add(\{f \in \mathcal{T} : time - \zeta \leq f.start < time\})$ 
7    $\theta_f \leftarrow \text{compute\_throughput}(G, \mathcal{K}, \mathcal{R})$ 
8   for flow  $f$  in  $\mathcal{K}$  do
9      $f.sent \leftarrow \min(f.sent + \zeta \theta_f, f.size)$ 
10    if  $f.sent = f.size$  then
11       $\mathcal{K}.remove(f)$ 
12      if  $f \in \mathcal{I}$  then
13         $\beta_l.append(\frac{f.size}{f.dur})$ 
14      end
15    end
16  end
17 end
18 return  $\beta_l$ 

```

predictable since they do not stay in the network long enough to be affected by network variations. Thus, a simple empirical model is sufficient to estimate their FCT distribution.

SWARM finds a short flow's FCT distribution (6) by estimating (a) its RTT and (b) the number of RTTs to deliver its demand. A short flow's FCT is equal to the average duration of RTTs multiplied by the required number of RTTs. We derive the distribution of each metric separately and then combine them to compute the FCT distribution.

We compute the distribution of the number of RTTs to deliver a flow's demand by conducting offline experiments in a small testbed. We repeat the experiments for different configurations (flow size, slow start threshold, initial congestion window) and network settings (drop rates, RTT) and store the results in a table that maps the setting to the measured distribution.

Next, we estimate the RTT, which is equal to the sum of the propagation delay (a constant determined by the flow's path) and the queueing delay along the flow's path. To estimate queueing delay, we collect data from sending small flows on links with different utilization and active flow counts (§B).

SWARM combines these distributions to derive the FCT distribution for short flows. It multiplies the number of RTTs by the sum of the propagation delay and queueing delay.

3.4 Expressivity, Scaling, and Robustness

Expressivity. SWARM supports any failure or mitigation as long as we can model it as changes to the network state or the traffic. SWARM does not need the root cause of a failure or details of a mitigation but only needs to understand their observable impact (*e.g.*, packet drop, port down). This flexibility allows SWARM to support a wide range of failures and mitigations (Table 2). Existing production and state-of-the-art systems [63, 71] do not support many of these failures, while cloud providers [7, 24, 31, 55, 63, 71] commonly observe them.

Robustness. SWARM’s network-level mitigations should ideally mask network failures. However, this masking is not always perfect, and a cloud service might react to a mitigation (or the failure in general) by changing the traffic demand (*e.g.*, using retries). For this reason, SWARM does not model a fixed traffic demand. Instead, it draws multiple traffic samples from the historical distribution of flow arrivals, sizes, and communication probabilities to ensure statistical significance (§3.3). When operators do not have such statistics (*e.g.*, after a previously unseen failure or a data center expansion), SWARM uses a distribution that captures maximum uncertainty [51].

SWARM carefully models uncertainty, but its CLP estimates may not align with what operators observe after installing the mitigation. This can happen if an uncommon failure manifests itself that was not captured during the sampling process or if the traffic changes and the historical distribution is not representative anymore. In such instances, the auto-mitigation system that uses SWARM must update its inputs and invoke SWARM again to revise the mitigation. In other words, mitigation does not have to be a single-shot process and can be adjusted over time, especially since failure diagnosis might take hours to days [21, 63].

Scaling. SWARM must find a mitigation action quickly, even on large clusters. We scale it using the following techniques:

An ultra-fast max-min fair computation algorithm. We use an approximate computation of network-wide max-min fair share rates [45], which provides significant speedup over the state-of-the-art methods [34] without affecting quality.

Efficient network state and traffic update. The traffic trace is independent of the network state. This means SWARM can compute the traffic samples offline. However, when SWARM invokes CLPEstimator, it must update network state to reflect each mitigation and then re-compute routing samples. To scale, SWARM separates the topology representation from the traffic representation. It models the former as a graph (§3.3) and sorts the latter into a list of tuples (source and destination server, flow size, and flow start time). This design enables, for example, disabling a link by changing the drop rate in G to 100% or updating the traffic if a mitigation such as VM migration modifies a flow.

Parallelism and pipelining. SWARM (a) evaluates demand and routing samples in parallel, and (b) parallelizes and

pipelines routing sample generation with epoch execution.

Reducing the number of epochs. After these steps, the bottleneck is the number of epochs in Alg. 1. We use two techniques. Firstly, we initialize on an already warmed-up network instead of starting from an empty network. This eliminates the need for a set of epochs at the beginning to mimic the cold-start effect. Secondly, the residual impact of flows that compete within an epoch diminishes over time. Therefore, epochs with large time differences present independent network snapshots. This observation allows us to compute their CLPs independently in parallel and combine the results to form an overall distribution.

Traffic downscaling. Following POP [47], SWARM down-scales the demand matrix with minimal impact on throughput. It splits a network with link capacity c into k sub-networks with link capacity $\frac{c}{k}$ and divides traffic randomly across these sub-networks. POP [47] recommends choosing a value for k that is much smaller than the number of flows, which allows each partition to capture the network contention. This approach works with any flow arrival distribution. We use Poisson distributions [23, 35, 41], where assigning flows randomly is the same as downscaling the arrival rate based on the Poisson splitting property.

4 Evaluation

Our prototype of SWARM has 1500 lines of Python. We evaluate it on three categories of incidents and show it outperforms existing methods. We also show it scales and finds the best mitigation for a 16K-server topology within 5 minutes.

4.1 Methodology

Metric. We evaluate each approach by computing the *Performance Penalty* (%), which is the relative difference between the CLP metrics that result from the best possible mitigation and the one each technique suggests. This metric captures the unnecessary performance degradation caused by a technique choosing sub-optimal mitigation. Often, the difference between the best mitigation and the “runner-up” is insignificant, resulting in a small penalty. In contrast, the difference between the best mitigation and the one the baselines choose can be as high as 200%.

Experimentation setup. We evaluate SWARM using Mininet [37], NS3 [29], and a physical testbed. Here is the summary of our setup (see §C for more details):

Traffic characterization. We use Azure production logs to derive the flow inter-arrival time, a commonly used distribution from DCTCP [5] for our flow size distribution, and [38] for server-to-server communication probability. We ensure that the trace duration is long enough to capture all flow sizes and that we do not capture an empty network’s effect. Finally, we run on 30 different traces to ensure robustness.

Emulation Setup. We mainly use Mininet for our evaluation since it leverages a real TCP/IP stack from the Linux

kernel. We extend it to improve the fidelity of our results (monitoring systems, queueing disciplines). We report results on 57 scenarios (over 4000 hours of experiment) across 3 types of common incidents in cloud providers [7, 24, 55, 63, 71] (Table 2). We describe these scenarios in §4.2 and Table A.1. We use the Clos topology from Figure 2.

We seek to emulate 1500 flows arriving per second per server, which results in 12,000 flows every second. These are on links with 40 Gbps bandwidth and 50 μ s propagation delay. However, running Mininet on a VM with 64 cores and 256 GB cannot emulate this demand. Instead, we use [48, 50] to downscale the traffic and link capacities by 120 $\times$ (see §C.4). We run each emulation for 500 s and measure the performance for flows that start within [50, 150] s to avoid capturing effects from an empty network. We report results for both Cubic [28] and BBR [12] (and DCTCP [5] in our simulation).

Simulation Setup. We use a Clos topology with 128 servers, 32 ToRs, 32 T1s, and 16 T2s, all connected by 20 Gbps, 100 μ s links. We use DCTCP [5] as the congestion control algorithm to show generality. Each of our traffic traces is 10 s long, and we measure the flows that start within [0.5, 1) s⁴. We use DCTCP [5] and FbHadoop [54] flow size distributions.

Testbed Setup. We use a different variant of Clos (see §C.3) with 32 servers, six TORs, four T1s, and two T2 switches, all connected by 10 Gbps 200 μ s links. All switches are Arista 7050QX-32. We introduce random packet loss in the testbed using a user-defined access control list (ACL) to match bits on the IP ID field in packet headers and directly modify the Broadcom firmware to drop packets the ACL matches. Thus, packet loss rates in the testbed are powers of two based on the number of bits the ACL matches on. We evaluate the impact of the failure and each mitigating action on the testbed with a traffic load of 3000 flows per second. Each trace is 30 seconds long, and we measure across flows that start within [2, 5) s.

SWARM Parameters. We use 32 different random traffic traces and 1000 routing samples based on §3.3. For our baseline comparisons (with Mininet), each trace is 200 s long. We compute the performance over all the flows that start within [50, 150] s. Each epoch is 200 ms. We also consider any flow with a size $\leq$ 150 KB short.

Baselines. We compare SWARM to:

NetPilot [63] iterates through each possible mitigation, computes the maximum link utilization, and picks the action that minimizes utilization. NetPilot does not model link utilization on faulty links, so it always disables corrupted links. We report these results as NetPilot-Orig. We also extend NetPilot to mitigate only if the resulting maximum link utilization is below a threshold. We report these results as NetPilot-80 for an 80% and NetPilot-99 for a 99% utilization threshold.

⁴We adjust the duration based on the flow arrival rate and link bandwidths. When these values are smaller as in our Mininet experiments, it will take longer for a certain number of flows to arrive, and each flow remains in the network for a longer period. Thus, we need to cut a longer duration at the beginning to ensure we are not capturing the effect of the empty network.

CorrOpt [71] only considers link corruption failures. It disables a link if the number of remaining paths to the spine after taking the action is above a threshold. We consider three thresholds: CorrOpt-25, CorrOpt-50, and CorrOpt-75, which use a threshold of 25%, 50%, and 75%, respectively.

Operator playbooks. When an FCS error occurs above the ToR where there is path redundancy, the Azure playbook will disable the affected link if the number of remaining up-links at the switch is above a certain threshold. We consider three thresholds: Operator-25 uses a 25% threshold, Operator-50 uses 50%, and Operator-75 uses 75%. When there is packet loss of more than 10^{-3} at or below the ToR, the playbook will drain the affected nodes, which is expensive and risks VM reboots or interrupts. Otherwise, it would take no action.

Some baselines cause a partitioned network under certain scenarios in part due to the smaller scale in our Mininet evaluations. Unless noted otherwise, we only report cases where all baselines keep the network connected for a fair comparison.

Comparators. Most experiments use two priority comparators (§3.2) (§D.4 shows SWARM achieves low penalty across two other comparators including a linear one as well):

PriorityFCT minimizes the 99p FCT. It uses two tiebreakers, 1p throughput followed by average throughput.

PriorityAvgT maximizes the average throughput first, using two tiebreakers, 99p FCT, followed by 1p throughput.

Two mitigations are tied on a particular metric if they are within 10% of each other on that metric.

4.2 Baseline Comparisons

We evaluate SWARM over three different failure scenarios, which are common in production incidents at Azure.

Scenario 1: Link-level packet corruption with network redundancy. In this scenario, we evaluate different combinations of two links consecutively experiencing FCS errors with a drop rate of $\sim$ 5% (*high*) or $\sim$ 0.005% (*low*). This is the most common failure pattern at Azure, and both drop rates are detected and reported as incidents. In this case, the viable mitigations are doing nothing, disabling the link, undoing past mitigations, changing WCMP weights, or any feasible combination of these. All baselines support this scenario because the link failure is above the ToR, and there is path redundancy.

In Fig. 7, we compare the performance penalty of SWARM to the baselines for both comparators. We do not compare to NetPilot-orig in this scenario since it partitions the network in 16 out of 32 failure pairs, and the results on the remaining failures are not statistically significant. We use violin plots to show the performance penalties' distribution across all incidents for each candidate approach and each metric. A tall violin plot indicates that performance penalties span a wide range, while a short and wide plot indicates that penalties are clustered within a small set of values. Even though the baselines do not explicitly use comparators, the best mitigation depends on the comparator, which causes the penalty to change for the baselines across different comparators.

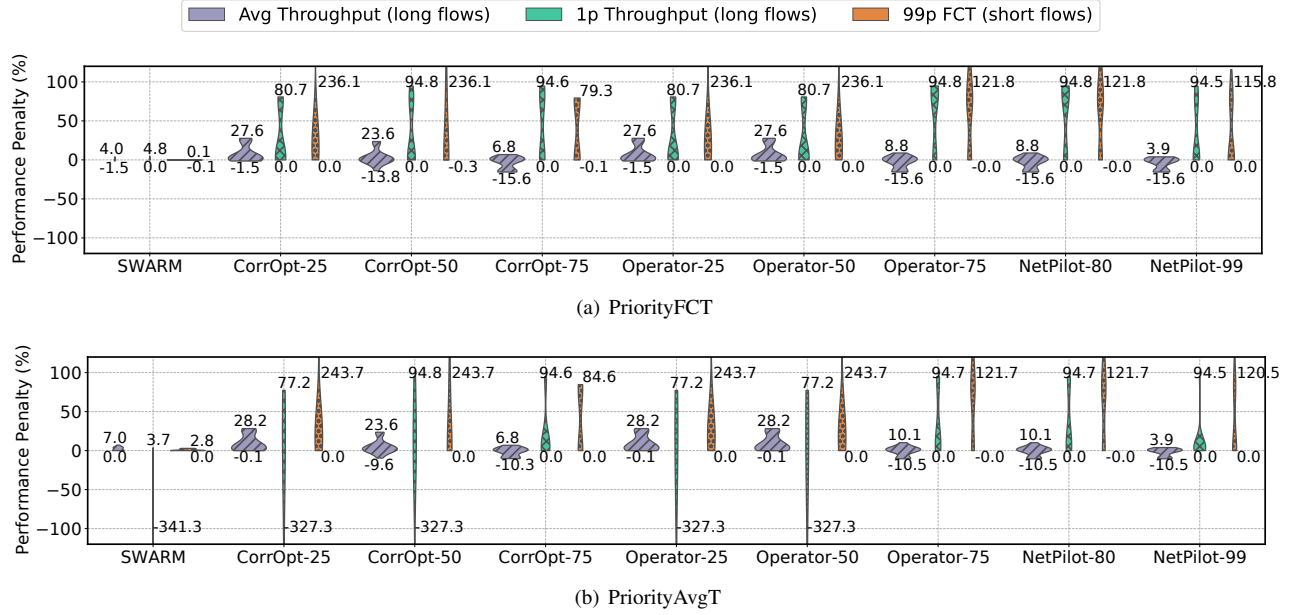


FIGURE 7: Comparison of SWARM and other baselines under Scenario 1 in Mininet. SWARM achieves $793\times$ lower performance penalty on 99p FCT in the worst case compared to the next best baseline on PriorityFCT. SWARM is the only technique that achieves near-optimal performance across all three metrics and performs equally well across both comparators.

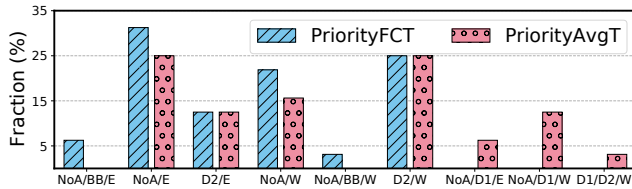


FIGURE 8: SWARM's actions in Scenario 1 (see §4). It chooses from nine action combinations and decides to take no action in more than 25% of the time. (NoA: No Action on link 2, D2: Disable link 2, BB: Bring Back link 1, D1: Disable link 1, W: WCMP Routing, E: ECMP Routing)

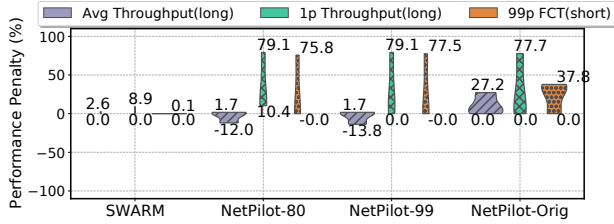
SWARM outperforms all baselines with a performance penalty consistently close to zero. When using the Priority-FCT comparator, SWARM has a maximum FCT performance penalty of 0.1%, compared to 79.3% penalty of the closest baseline, CorrOpt-75 (Fig. 7(a)). For the Priority-AvgT comparator, SWARM's maximum penalty on average throughput is similar to several of the baselines, such as NetPilot-99 and CorrOpt-75. However, SWARM reduces the performance penalty across *all three* CLP metrics while the baselines suffer from high performance penalties across at least one (Fig. 7(b)). SWARM's superior performance is due to its ability to analyze a broader set of mitigations and to choose one that reduces performance impact on all CLP metrics (see §F).

The negative penalty in some cases is because of the inherent trade-off between different metrics. For instance, the primary objective in Fig. 7(a) is to minimize short flows' 99p FCT. The optimal mitigation is the one that has the best FCT,

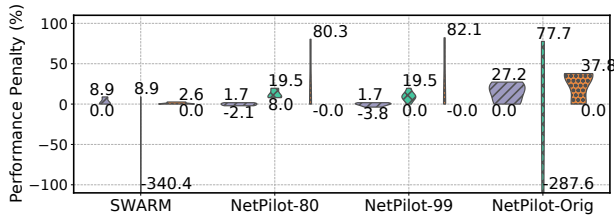
even if it worsens other metrics. For example, CorrOpt-75 selects actions that increase the 99p short flow's FCT (highest priority) by up to 80% compared to the optimal action but achieve better average throughput (lower priority).

We show the diversity of SWARM's proposed mitigation under each comparator in Fig. 8. We focus on the mitigation for the second failure since the action space is larger and includes options such as bringing back a previously disabled link. We find SWARM chooses from nine different possible mitigations and decides to take no action in more than 25% of the cases. In two scenarios under PriorityFCT, it not only takes no action on the second failure but also *reinstates* the faulty link it previously disabled from the first failure (action NoA/BB/E). SWARM considers the failure locations, their intensity, and the traffic demands and decides to preserve those links rather than eliminate their capacity and cause congestion. Under PriorityAvgT, SWARM chooses combinations of mitigations such as disabling both links and adjusting WCMP weights.

Scenario 2: Congestion on a link. Operators shut down several faulty links (prior failures), causing the network to become over-subscribed. Meanwhile, an aggregation-core layer starts operating at half capacity (due to fiber cuts). CorrOpt and operator playbooks do not support this as they ignore traffic dynamics. NetPilot can reason about congestion but assumes the rest of the network is under-utilized. We evaluate SWARM and NetPilot under two failure patterns: (i) where the network is under-utilized and (ii) where a second link drops packets and reduces network capacity (Table A.1).



(a) PriorityFCT



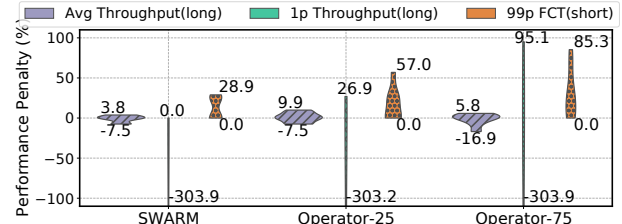
(b) PriorityAvgT

FIGURE 9: Comparison of SWARM and NetPilot variants under Scenario 2 in Mininet. Under the PriorityFCT comparator, SWARM always chooses a mitigation with near-optimal FCT performance ($\leq 0.1\%$), while the next best approach chooses mitigations up to 37.8% worse than optimal. SWARM is also the only approach with low performance penalty across all three metrics.

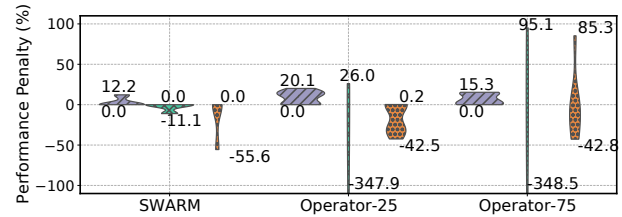
Fig. 9 compares the performance penalty of SWARM to the NetPilot variants for both comparators across both failure patterns. SWARM achieves consistently low penalty for its target CLP metric: 99p FCT in Fig. 9(a) and average throughput in Fig. 9(b). Since NetPilot assumes the rest of the network is under-utilized, it aggressively disables links and causes a large performance impact. Under PriorityFCT, SWARM chooses a mitigation with near-optimal performance on FCT, while the next best approach suffers an FCT penalty of 38%. Under the PriorityAvgT comparator, the NetPilot-80/99 variants result in lower impact on average throughput, but at the cost of increased performance penalty in at least one other metric (*e.g.*, NetPilot-80 achieves 7.2% less penalty in terms of average throughput at the cost of 80% penalty on 99p FCT) while SWARM is the only technique that performs well across both comparators and all three metrics.

Scenario 3: Packet corruption at ToR. In this scenario, we consider (Table A.1) failure patterns in which a ToR drops packets at either *high* 5% or *low* 0.005% rates. We also consider cases where both the ToR and a core link drop packets. CorrOpt and NetPilot do not support this failure as they can only account for scenarios where the network has redundant paths. The operator playbook makes a local decision on whether to mitigate the failure based on the severity of the packet loss, without considering the overall network condition. When the second failure occurs above the ToR, it reduces the capacity in the network core, causing the operator’s approach to incur higher performance penalties.

Both playbook-based approaches suffer from performance



(a) Priority FCT



(b) Priority AvgT

FIGURE 10: Comparison of SWARM and operator playbooks on Scenario 3 in Mininet. Under PriorityFCT, SWARM has $2\times$ lower 99p FCT penalty in the worst-case compared to operator playbooks. It also protects other non-priority metrics while optimizing for the priority metric (it outperforms baselines in both average throughput and 99p FCT under PriorityAvgT).

penalties at least $2\times$ higher than SWARM under PriorityFCT (Fig. 10). SWARM has a worst-case FCT penalty of 28.9% while the best operator approach has a worst-case FCT penalty of 57%. SWARM is again the only approach that achieves low penalty across all three metrics for both comparators.

4.3 Other Results

Scalability. In Fig. 11(a), we show the time SWARM needs to find the best mitigation. SWARM’s runtime scales linearly with the number of servers. Even on large-scale Clos topologies with 16K servers, SWARM needs less than 5 minutes, a fraction of the mitigation times operators report today [1, 21].

SWARM employs several approximation techniques to scale §3.4. In Fig. 11, we quantify the error and speed-up of these methods compared to a version of SWARM without these approximations. Each technique in §3.4 contributes significantly to speedup: (a) the max-min fair algorithm improves run-time by $36.3\times$ and only introduces $\leq 0.9\%$ error; (b) downscaling traffic by $2\times$ does not introduce additional error but produces $73.6\times$ speedup! (c) adding warm-start and reducing the number of epochs results in $105.7\times$ speedup and $\leq 1.2\%$ error. Future work can further speed this up by increasing the scale factor or reducing the number of epochs.

Sensitivity analysis. We evaluate SWARM’s sensitivity to various inputs, including the drop rate estimates and the flow-arrival rates (see §D.1 for details).

As the inputs to the system vary, there are a few inflection points where SWARM is sensitive. These points are where SWARM can make mistakes and pick sub-optimal mitigations

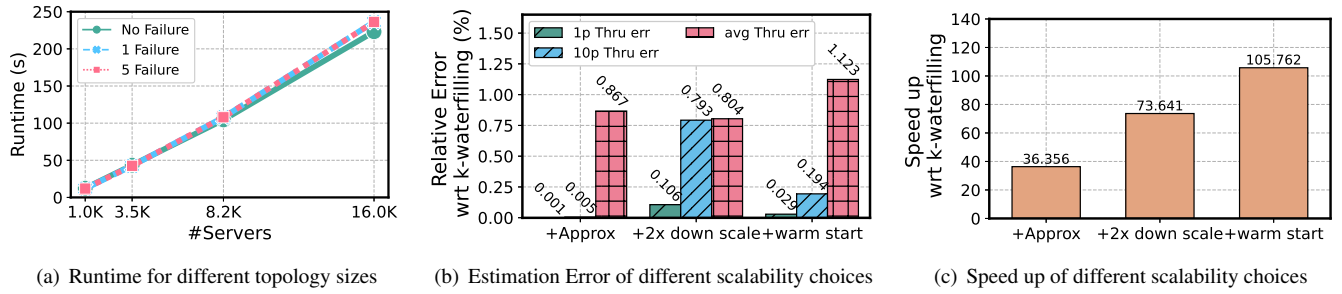


FIGURE 11: Scalability. (a) SWARM scales almost linearly with the number of servers. (b) Error introduced by each of our scaling techniques (approx refers to the ultra-fast max-min fair algorithm). (c) Speed-up from each of our scaling components. We compare with a version of SWARM that does not use the methods from §3.4, instead relies on extended 1-waterfilling [34] to compute max-min fair rates.

if the inputs are noisy. However, we find the difference between the impact of the mitigations is small around these inputs. Outside of these areas, the choice of better mitigation is clear. Thus, SWARM can tolerate large errors in the input distribution: the difference between mitigations is either large enough around that point so the choice of mitigation is clear or it is small enough where a mistake is not too costly.

SWARM can also pick the best mitigation under different congestion control protocols (see Fig. A.3). For this, we compare two protocols with different behaviors under loss: (1) Cubic [28], which drastically reduces its sending rate under packet loss, and (2) BBR [12], which does not. SWARM picks the best mitigation irrespective of which protocol we use. However, its approximations of the 1p throughput distribution are more accurate when the mix of protocols is known (it can explicitly account for their differences in handling loss).

Simulation validation. To show SWARM’s effectiveness at larger scales with realistic link speeds and latencies, we conducted a simulation using NS3 [29] on a 128-server topology with 20 Gbps 100 μ s links. NS3 alone takes over a day to complete one simulation run (one sample). We invested significant effort in parallelizing NS3 using MPI and reduced the time to run one sample to 6 hours. On this topology, we induce a failure in which two links drop packets (one ToR-T1 at 0.005% and one T1-T2 at 0.5%). This scenario shows the complex effect of different packet drops at different levels of datacenters and the trade-off between causing congestion by disabling the links versus incurring packet drops by taking no action. Apart from the DCTCP distribution, we also simulated FbHadoop [54] that has more short flows. These experiments required over 2100 hours to complete.

Fig. 12 shows the performance penalties of different actions. SWARM is able to identify if the congestion introduced by disabling the link would impact the flows more than the packet drop or vice-versa. In contrast, prior work (NetPilot, CorrOpt, and Operator) ignores the impact of traffic and failure characteristics on the mitigation. SWARM finds the best mitigation (only disabling the high drop rate link). In contrast,

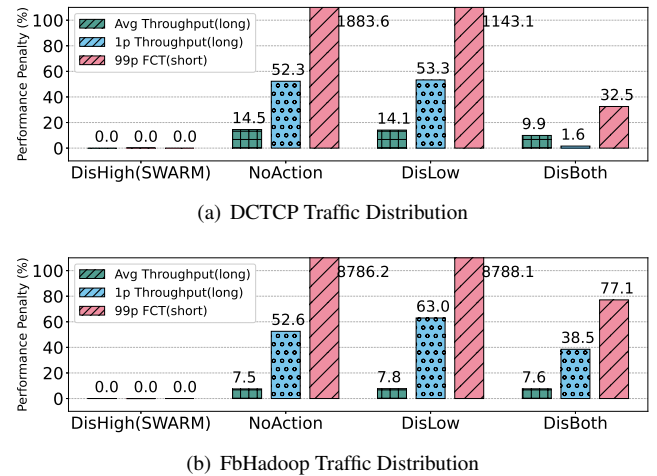


FIGURE 12: NS3 evaluation with different flow size distributions. (Dis=Disable, High=High drop link, Low=Low drop link)

the baselines either disable both links or keep both links and incur 32% – 78% penalty on 99p FCT. Note that the performance penalty of not taking any action is the same as only disabling the link with a low packet drop since they both keep the link with a high drop rate in the network. This link ends up dropping many packets and determines the tail FCT.

Testbed validation. To demonstrate SWARM makes high-quality decisions even on a physical testbed, we induce a failure pattern from Scenario 1 in which a ToR – T1 link randomly drops 6.25% ($\frac{1}{16}$) of the packets, while a link from a different T1 to a T2 also drops packets at 0.39% ($\frac{1}{256}$).

SWARM picks an optimal mitigation for Priority-FCT and a mitigation with less than 1% penalty for PriorityAvgT (Fig. 13). In contrast, the FCT performance penalty for choosing the worst action under PriorityFCT is over 1000%. While the average throughput performance penalty is low across all mitigations in this incident, SWARM picks an action with a low penalty across all three metrics under the PriorityAvgT comparator. It avoids the 93% penalty in 1p throughput and

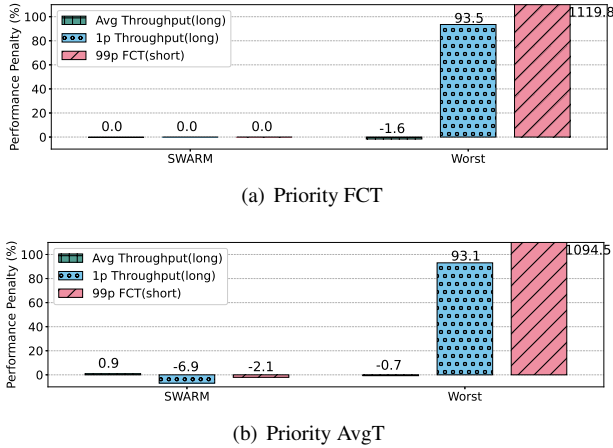


FIGURE 13: Physical testbed validation with two links dropping packets. We consider four mitigation strategies (disable or take no action). Optimal mitigation has zero penalty.

1095% penalty in 99p FCT for the worst action.

Impact of the comparator. We also observe that SWARM adjusts its decision based on the comparator. Across all the single link failures from Scenario 1 (Table A.1), SWARM takes no action more often under the PriorityAvgT comparator than the PriorityFCT. This is because average throughput is sensitive to the remaining capacity in the network, so SWARM is more likely to keep the lossy link up.

Justifying design choices. We also conducted several experiments to justify the special treatment of loss-limited flows, estimating distributions of FCT and throughput, using multiple epochs to capture flow dynamics, using distributional measures to determine the best mitigation, and the importance of accounting for queueing delay (see §D.3).

5 Discussion and Limitations

SWARM is a first step towards CLP-aware incident mitigation. It has limitations that future work can address.

Support for loss-less transport. Cloud providers increasingly use RDMA [22, 58] for internal traffic. We can modify SWARM’s CLPEstimator to (a) detect and account for pauses and (b) model loss-recovery approaches when losses happen.

Support for other routing protocols. SWARM applies to any datacenter that relies on ECMP or WCMP for routing. These are commonly used in Clos topologies. Our experiments show the effectiveness of SWARM on different variants of Clos. Future work can extend this to topologies that use traffic engineering (e.g., direct-connect [49, 61])

Mitigations with transient effects. Some actions introduce transient risk. For example, a switch may drop packets during reboot, causing a non-steady-state impact on long flows.

Approximate failure localization. SWARM waits for operators or automation to localize the failure. It can instead use a spatial failure distribution which is available much sooner. This lowers the mean time to repair. SWARM relies on the

correct input of the *type* of failure.

Impact on wide area network traffic. WAN traffic is a small fraction of the datacenter traffic [10, 19], so we ignore its impact. When WAN traffic increases, we would need to account for larger RTT.

Other extensions. Future work can extend SWARM to other metrics such as jitter and failures of software components such as load balancers. It can also account for estimated repair time in ranking mitigations, which can be challenging as incidents with vastly different repair times often have similar symptoms [25]. Future work can also explore the benefits of modeling more than two classes (short and long) of flows.

6 Related Work

Impact or risk estimation in networking. No prior work considers CLP-aware failure mitigation but several estimate the impact or future risk of management operations. Janus [4] focuses on risk estimation for datacenter management operations. RSS [64] estimates the risk for backbone management. TEAVAR [11] and [17, 62] route traffic in production wide area networks to minimize long-term impact. [43] models the risk of demand uncertainty and its impact on revenue in a WAN. Other approaches minimize the impact of failures without explicitly accounting for risk [39, 69].

NetPilot [63] and CorrOpt [71] are closest to SWARM. These can fail to produce mitigations with minimal CLP impact (see §2) because they (a) ignore the failure pattern, (b) do not account for traffic changes, (c) use proxy metrics (e.g., maximum utilization) that only loosely correlate with CLPs, and (d) estimate impact on a healthy network, which limits the set of mitigations they support.

Computing fair share. Prior work assumes flows follow max-min fairness, which matches the objective of TCP [20], and either employ optimization [16, 30, 45, 46, 59], use simulations [68], or iterative algorithms [15, 32, 34, 45, 52, 53] to find the fair share. These approaches are limited; they assume flow rates are only limited by the network capacity and ignore packet drops.

7 Conclusion

Failure mitigation is crucial in running datacenters at scale, but operators often find it hard to choose the right action. SWARM ranks mitigations based on their impact on well-known CLP metrics. Our evaluation shows approximating these metrics is possible at scale and results in mitigations with orders of magnitude lower performance penalty.

Acknowledgments. We thank our shepherd, Sergey Gorinsky, and the anonymous reviewers for their insightful comments. We also thank Ricardo Bianchini, Ranveer Chandra, Jitu Padhye, Solal Pirelli, Rachee Singh and Lihua Yuan for their helpful feedback. This material is based upon work supported in part by the U.S. National Science Foundation under grant No. CNS-1901523.

References

- [1] Google cloud incident summary. <https://status.cloud.google.com/summary>.
- [2] Rca - network latency issue – west europe (tracking id 8klc-1t8). <https://status.azure.com/en-us/status/history/>.
- [3] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. In *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication, SIGCOMM '08*, page 63–74, New York, NY, USA, 2008. Association for Computing Machinery.
- [4] Omid Alipourfard, Jiaqi Gao, Jeremie Koenig, Chris Harshaw, Amin Vahdat, and Minlan Yu. Risk based planning of network changes in evolving data centers. In *ACM SOSP*, 2019.
- [5] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. Data center tcp (dctcp). In *ACM SIGCOMM*, 2010.
- [6] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. pfabric: minimal near-optimal datacenter transport. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, 2013.
- [7] Behnaz Arzani, Selim Ciraci, Luiz Chamon, Yibo Zhu, Hongqiang Harry Liu, Jitu Padhye, Boon Thau Loo, and Geoff Outhred. 007: Democratically finding the cause of packet drops. In *15th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 18)*, pages 419–435, 2018.
- [8] Behnaz Arzani, Selim Ciraci, Boon Thau Loo, Assaf Schuster, and Geoff Outhred. Taking the blame game out of data centers operations with netpoirot. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 440–453, 2016.
- [9] Behnaz Arzani, Selim Ciraci, Stefan Saroiu, Alec Wolman, Jack Stokes, Geoff Outhred, and Lechao Diwu. Privateeye: Scalable and privacy-preserving compromise detection in the cloud. In *NSDI 20*, 2020.
- [10] Theophilus Benson, Aditya Akella, and David A Maltz. Network traffic characteristics of data centers in the wild. In *Proceedings of the 10th ACM SIGCOMM conference on Internet measurement*, pages 267–280, 2010.
- [11] Jeremy Bogle, Nikhil Bhatia, Manya Ghobadi, Ishai Menache, Nikolaj Bjørner, Asaf Valadarsky, and Michael Schapira. Teavar: Striking the right utilization-availability balance in wan traffic engineering. In *ACM SIGCOMM*, 2019.
- [12] Neal Cardwell, Yuchung Cheng, C. Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. Bbr: Congestion-based congestion control. *ACM Queue*, 14, September-October:20 – 53, 2016.
- [13] Jeffrey D Case, Mark Fedor, Martin L Schoffstall, and James Davin. Simple network management protocol (snmp). Technical report, 1989.
- [14] Graham Cormode, Minos Garofalakis, Peter J. Haas, and Chris Jermaine. Synopses for massive data: Samples, histograms, wavelets, sketches. *Found. Trends Databases*, 4(1–3):1–294, jan 2012.
- [15] Emilie Danna, Avinatan Hassidim, Haim Kaplan, Alok Kumar, Yishay Mansour, Danny Raz, and Michal Segalov. Upward max-min fairness. *J. ACM*, 64(1), mar 2017.
- [16] Emilie Danna, Subhasree Mandal, and Arjun Singh. A practical algorithm for balancing the max-min fairness and throughput objectives in traffic engineering. In *2012 Proceedings IEEE INFOCOM*, pages 846–854, 2012.
- [17] Ferhat Dikbiyik, Massimo Tornatore, and Biswanath Mukherjee. Minimizing the risk from disaster failures in optical backbone networks. *Journal of Lightwave Technology*, 32(18):3175–3183, 2014.
- [18] A. Dvoretzky, J. Kiefer, and J. Wolfowitz. Asymptotic Minimax Character of the Sample Distribution Function and of the Classical Multinomial Estimator. *The Annals of Mathematical Statistics*, 27(3):642 – 669, 1956.
- [19] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, Vivek Bhanu, Adrian Caulfield, Eric Chung, et al. Azure accelerated networking: {SmartNICs} in the public cloud. In *USENIX Symposium on Networked Systems Design and Implementation*, pages 51–66, 2018.
- [20] S. Ben Fred, T. Bonald, A. Proutiere, G. Régnié, and J. W. Roberts. Statistical bandwidth sharing: A study of congestion at flow level. In *Proceedings of the 2001 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, SIGCOMM '01*, page 111–122, New York, NY, USA, 2001. Association for Computing Machinery.
- [21] Jiaqi Gao, Nofel Yaseen, Robert MacDavid, Felipe Vieira Frujeri, Vincent Liu, Ricardo Bianchini, Ramaswamy Aditya, Xiaohang Wang, Henry Lee, David Maltz, et al. Scouts: Improving the diagnosis process

through domain-customized incident routing. In *ACM SIGCOMM*, 2020.

- [22] Yixiao Gao, Qiang Li, Lingbo Tang, Yongqing Xi, Pengcheng Zhang, Wenwen Peng, Bo Li, Yaohui Wu, Shaozong Liu, Lei Yan, et al. When cloud storage meets {RDMA}. In *18th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 21)*, pages 519–533, 2021.
- [23] Monia Ghobadi, Ratul Mahajan, Amar Phanishayee, Nikhil Devanur, Janardhan Kulkarni, Gireeja Ranade, Pierre-Alexandre Blanche, Houshuo Rastegarfar, Madeleine Glick, and Daniel Kilper. Projector: Agile reconfigurable data center interconnect. In *Proceedings of the 2016 ACM SIGCOMM Conference, SIGCOMM '16*, page 216–229, New York, NY, USA, 2016. Association for Computing Machinery.
- [24] Phillipa Gill, Navendu Jain, and Nachiappan Nagappan. Understanding network failures in data centers: Measurement, analysis, and implications. *SIGCOMM Comput. Commun. Rev.*, 41(4):350–361, aug 2011.
- [25] Ramesh Govindan, Ina Minei, Mahesh Kallahalla, Bikash Koley, and Amin Vahdat. Evolve or die: High-availability design principles drawn from google's network infrastructure. In *Proceedings of the 2016 ACM SIGCOMM Conference, SIGCOMM '16*, page 58–72, New York, NY, USA, 2016. Association for Computing Machinery.
- [26] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. Multi-resource packing for cluster schedulers. In *Proceedings of the 2014 ACM Conference on SIGCOMM, SIGCOMM '14*, page 455–466, New York, NY, USA, 2014. Association for Computing Machinery.
- [27] Albert Greenberg, James R. Hamilton, Navendu Jain, Srikanth Kandula, Changhoon Kim, Parantap Lahiri, David A. Maltz, Parveen Patel, and Sudipta Sengupta. V12: A scalable and flexible data center network. In *ACM SIGCOMM*, 2009.
- [28] Sangtae Ha, Injong Rhee, and Lisong Xu. Cubic: a new tcp-friendly high-speed tcp variant. *ACM SIGOPS operating systems review*, 42(5):64–74, 2008.
- [29] Thomas R Henderson, Mathieu Lacage, George F Riley, Craig Dowell, and Joseph Kopena. Network simulations with the ns-3 simulator. *SIGCOMM demonstration*, 14(14):527, 2008.
- [30] Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill, Mohan Nanduri, and Roger Wattenhofer. Achieving high utilization with software-driven wan. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, pages 15–26, 2013.
- [31] Qun Huang, Haifeng Sun, Patrick PC Lee, Wei Bai, Feng Zhu, and Yungang Bao. Omnimon: Re-architecting network telemetry with resource efficiency and full accuracy. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 404–421, 2020.
- [32] Sushant Jain, Alok Kumar, Subhasree Mandal, Joon Ong, Leon Poutievski, Arjun Singh, Subbaiah Venkata, Jim Wanderer, Junlan Zhou, Min Zhu, Jon Zolla, Urs Hölzle, Stephen Stuart, and Amin Vahdat. B4: Experience with a globally-deployed software defined wan. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM, SIGCOMM '13*, page 3–14, New York, NY, USA, 2013. Association for Computing Machinery.
- [33] Jiajun Jiang, Weihai Lu, Junjie Chen, Qingwei Lin, Pu Zhao, Yu Kang, Hongyu Zhang, Yingfei Xiong, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. How to mitigate the incident? an effective troubleshooting guide recommendation technique for online service systems. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020*, page 1410–1420, New York, NY, USA, 2020. Association for Computing Machinery.
- [34] Lavanya Jose, Stephen Ibanez, Mohammad Alizadeh, and Nick McKeown. A distributed algorithm to calculate max-min fair rates without per-flow state. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 3(2):1–42, 2019.
- [35] Simon Kassing, Asaf Valadarsky, Gal Shahaf, Michael Schapira, and Ankit Singla. Beyond fat-trees without antennae, mirrors, and disco-balls. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication, SIGCOMM '17*, page 281–294, New York, NY, USA, 2017. Association for Computing Machinery.
- [36] Frank Kelly. Fairness and stability of end-to-end congestion control. *European journal of control*, 9(2-3):159–176, 2003.
- [37] Bob Lantz, Brandon Heller, and Nick McKeown. A network in a laptop: rapid prototyping for software-defined networks. In *ACM SIGCOMM Workshop on Hot Topics in Networks*, pages 1–6, 2010.
- [38] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming

- Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. Hpsc: high precision congestion control. In *Proceedings of the ACM Special Interest Group on Data Communication, SIGCOMM '19*, page 44–58, New York, NY, USA, 2019. Association for Computing Machinery.
- [39] Hongqiang Harry Liu, Srikanth Kandula, Ratul Mahajan, Ming Zhang, and David Gelernter. Traffic engineering with forward fault correction. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, pages 527–538, 2014.
- [40] Chris Lonvick. The bsd syslog protocol. Technical report, 2001.
- [41] William M. Mellette, Rajdeep Das, Yibo Guo, Rob McGuinness, Alex C. Snoeren, and George Porter. Expanding across time to deliver bandwidth efficiency and low latency. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 1–18, Santa Clara, CA, February 2020. USENIX Association.
- [42] Marco Mellia and Hui Zhang. Tcp model for short lived flows. *IEEE communications letters*, 6(2):85–87, 2002.
- [43] D. Mitra and Qiong Wang. Stochastic traffic engineering for demand uncertainty and risk-aware network revenue management. *IEEE/ACM Transactions on Networking*, 13(2):221–233, 2005.
- [44] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. Homa: A receiver-driven low-latency transport protocol using network priorities. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pages 221–235, 2018.
- [45] Pooria Namyar, Behnaz Arzani, Srikanth Kandula, Santiago Segarra, Daniel Crankshaw, Umesh Krishnaswamy, Ramesh Govindan, and Himanshu Raj. Solving Max-Min Fair Resource Allocations Quickly on Large Graphs. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, 2024.
- [46] Pooria Namyar, Sucha Supittayapornpong, Mingyang Zhang, Minlan Yu, and Ramesh Govindan. A throughput-centric view of the performance of data-center topologies. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference, SIGCOMM '21*, page 349–369, New York, NY, USA, 2021. Association for Computing Machinery.
- [47] Deepak Narayanan, Fiodar Kazhamiaka, Firas Abuzaid, Peter Kraft, Akshay Agrawal, Srikanth Kandula, Stephen Boyd, and Matei Zaharia. Solving large-scale granular resource allocation problems efficiently with pop. SOSP '21, page 521–537, New York, NY, USA, 2021. Association for Computing Machinery.
- [48] R. Pan, Balaji Prabhakar, K. Psounis, and D. Wischik. Shrink: A method for scaleable performance prediction and efficient network simulation. In *IEEE INFOCOM 2003. Twenty-second Annual Joint Conference of the IEEE Computer and Communications Societies (IEEE Cat. No.03CH37428)*, volume 3, pages 1943–1953 vol.3, 2003.
- [49] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, Rishi Kapoor, Stephen Kratzer, Nanfang Li, Hong Liu, Karthik Nagaraj, Jason Ornstein, Samir Sawhney, Ryohei Urata, Lorenzo Vicisano, Kevin Yasumura, Shidong Zhang, Junlan Zhou, and Amin Vahdat. Jupiter evolving: transforming google’s datacenter network via optical circuit switches and software-defined networking. In *ACM SIGCOMM*, 2022.
- [50] Konstantinos Psounis, Rong Pan, Balaji Prabhakar, and Damon Wischik. The scaling hypothesis: Simplifying the prediction of network performance using scaled-down simulations. *SIGCOMM Comput. Commun. Rev.*, 33(1):35–40, jan 2003.
- [51] Derek W Robinson. Entropy and uncertainty. *Entropy*, 10(4):493–506, 2008.
- [52] Jordi Ros-Giralt, Noah Amsel, Sruthi Yellamraju, James Ezick, Richard Lethin, Yuang Jiang, Aosong Feng, Leandros Tassioulas, Zhenguo Wu, Min Yee Teh, and Keren Bergman. Designing data center networks using bottleneck structures. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference, SIGCOMM '21*, page 319–348, New York, NY, USA, 2021. Association for Computing Machinery.
- [53] Jordi Ros-Giralt and Wei Kang Tsai. A theory of convergence order of maxmin rate allocation and an optimal protocol. In *Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society (Cat. No. 01CH37213)*, volume 2, pages 717–726. IEEE, 2001.
- [54] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C. Snoeren. Inside the social network’s (datacenter) network. *SIGCOMM Comput. Commun. Rev.*, 2015.
- [55] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, and Alex C Snoeren. Passive realtime datacenter fault detection and localization. In *14th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 17)*, pages 595–612, 2017.

- [56] Sergey Sarykalin, Gaia Serraino, and Stan Uryasev. Value-at-risk vs. conditional value-at-risk in risk management and optimization. In *State-of-the-art decision-making tools in the information-intensive age*, pages 270–294. Informa, 2008.
- [57] Manish Shetty, Chetan Bansal, Sai Pramod Upadhyayula, Arjun Radhakrishna, and Anurag Gupta. Autotsg: learning and synthesis for incident troubleshooting. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*, page 1477–1488, New York, NY, USA, 2022. Association for Computing Machinery.
- [58] Arjun Singhvi, Aditya Akella, Dan Gibson, Thomas F Wenisch, Monica Wong-Chan, Sean Clark, Milo MK Martin, Moray McLaren, Prashant Chandra, Rob Cauble, et al. Irma: Re-envisioning remote memory access for multi-tenant datacenters. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 708–721, 2020.
- [59] Ankit Singla, Chi-Yao Hong, Lucian Popa, and P. Brighten Godfrey. Jellyfish: Networking data centers randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 225–238, San Jose, CA, April 2012. USENIX Association.
- [60] Sucha Supittayapornpong, Pooria Namyar, Mingyang Zhang, Minlan Yu, and Ramesh Govindan. Optimal oblivious routing for structured networks. In *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*, pages 1988–1997, 2022.
- [61] Min Yee Teh, Shizhen Zhao, Peirui Cao, and Keren Bergman. Couder: robust topology engineering for optical circuit switched data center networks. *arXiv preprint arXiv:2010.00090*, 2020.
- [62] Bruno Vidalenc, Laurent Ciavaglia, Ludovic Noirie, and Eric Renault. Dynamic risk-aware routing for ospf networks. In *2013 IFIP/IEEE International Symposium on Integrated Network Management (IM 2013)*, pages 226–234, 2013.
- [63] Xin Wu, Daniel Turner, Chao-Chih Chen, David A. Maltz, Xiaowei Yang, Lihua Yuan, and Ming Zhang. Netpilot: Automating datacenter network failure mitigation. In *ACM SIGCOMM*, 2012.
- [64] Yiting Xia, Ying Zhang, Zhizhen Zhong, Guanqing Yan, Chiun Lin Lim, Satyajeet Singh Ahuja, Soshant Bali, Alexander Nikolaidis, Kimia Ghobadi, and Manya Ghobadi. A social network under social distancing: Risk-Driven backbone management during COVID-19 and beyond. In *NSDI 21*.
- [65] Nofel Yaseen, John Sonchack, and Vincent Liu. tpprof: A network traffic pattern profiler. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 1015–1030, Santa Clara, CA, February 2020. USENIX Association.
- [66] Qiao Zhang, Guo Yu, Chuanxiong Guo, Yingnong Dang, Nick Swanson, Xinsheng Yang, Randolph Yao, Murali Chintalapati, Arvind Krishnamurthy, and Thomas Anderson. Deepview: Virtual disk failure diagnosis and pattern detection for azure. In *Proceedings of the 15th USENIX Conference on Networked Systems Design and Implementation, NSDI’18*, page 519–532, USA, 2018. USENIX Association.
- [67] Qizhen Zhang, Kelvin KW Ng, Charles Kazer, Shen Yan, João Sedoc, and Vincent Liu. Mimicnet: fast performance estimates for data center networks with machine learning. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 287–304, 2021.
- [68] Kevin Zhao, Prateesh Goyal, Mohammad Alizadeh, and Thomas E Anderson. Scalable tail latency estimation for data center networks. *arXiv preprint arXiv:2205.01234*, 2022.
- [69] Zhizhen Zhong, Manya Ghobadi, Alaa Khaddaj, Jonathan Leach, Yiting Xia, and Ying Zhang. Arrow: restoration-aware traffic engineering. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 560–579, 2021.
- [70] Junlan Zhou, Malveeka Tewari, Min Zhu, Abdul Kabani, Leon Poutievski, Arjun Singh, and Amin Vahdat. Wcmp: Weighted cost multipathing for improved fairness in data centers. In *Proceedings of the Ninth European Conference on Computer Systems*, pages 1–14, 2014.
- [71] Danyang Zhuo, Monia Ghobadi, Ratul Mahajan, Klaus-Tycho Förster, Arvind Krishnamurthy, and Thomas Anderson. Understanding and mitigating packet corruption in data center networks. In *ACM SIGCOMM*, 2017.

A Algorithm Details

A.1 CLPEstimator Function

Alg. A.1 shows the CLPEstimator in detail. It first uses the DKW inequality [18] to compute the necessary number of samples to achieve a certain confidence level. It then adjusts the topology and the traffic based on the mitigation $\mathcal{M}$. Finally, it divides the resulting traffic into short and long flows, and computes their CLPs separately on multiple routing samples (§3.3).

Algorithm A.1: CLPEstimator Function.

Input: $\mathcal{T}$. traffic matrix (src, dest, start time, size).
Input: G . network state (location, type, failures).
Input: $\mathcal{M}$. mitigation.
Input: α . confidence threshold
Output: $\beta_l = \{\beta_l^{\mathcal{R}}\}$. $\beta_l^{\mathcal{R}}$ is the distribution of impact over all the long flows for routing sample $\mathcal{R}$.
Output: $\beta_s = \{\beta_s^{\mathcal{R}}\}$. $\beta_s^{\mathcal{R}}$ is the distribution of impact over all the short flows for routing sample $\mathcal{R}$.

```

1  $N \leftarrow \text{num\_samples}(\alpha)$ 
2  $G_a, \mathcal{T}_a \leftarrow \text{apply\_mitigation}(G, \mathcal{M}, \mathcal{T})$ 
3  $\mathcal{T}_s, \mathcal{T}_l \leftarrow \text{split\_traffic}(\mathcal{T}_a)$ 
4 for  $n \in 1 \dots N$  do
5    $\mathcal{R}_n \leftarrow \text{get\_routing\_sample}(G_a)$ 
6    $\beta_l^{\mathcal{R}_n} \leftarrow \text{impact\_long\_flows}(G_a, \mathcal{T}_l, \mathcal{R}_n)$ 
7    $\beta_s^{\mathcal{R}_n} \leftarrow \text{impact\_short\_flows}(G_a, \mathcal{T}_s, \mathcal{R}_n)$ 
8 end
9 return  $\beta_l, \beta_s$ 

```

A.2 Demand-Aware Max-Min Fair

CLPEstimator computes the impact of a given mitigation $\mathcal{M}$ on the throughput of long flows in two steps. (1) it estimates the drop-limited throughput for each flow (Line 1 in Alg. A.2), assuming there is no congestion and the packet drop enforces the maximum possible rate. (2) CLPEstimator computes the max-min fair rate of each flow and enforces these drop-limited throughputs as an upper limit on the rate (demand) of each flow (Line 2). This ensures that a flow does not receive more than its drop-limited throughput unless it is limited by its fair share when congestion is more severe than the drop rate.

Algorithms to compute max-min fairness [34] assume demands are unbounded and are only limited by the network capacities. In SWARM, we develop a demand-aware extension of these algorithms that can enforce limits on each flow's rate (Alg. A.3). To achieve this, we augment the topology and add one virtual edge per flow with capacity set at the drop-limited rate. Then, we use existing algorithms to compute the

Algorithm A.2: Compute throughput.

Input: G . current network state.
Input: $\mathcal{T}_{1 \times n}$. source destination pairs (long flows).
Input: $\mathcal{R}_{1 \times n}$. the (sampled) routing for each flow.
Output: F . max-min fair rate

```

1  $\theta \leftarrow \text{get\_drop\_limited\_throughput}(G, \mathcal{T}, \mathcal{R})$ 
2  $F \leftarrow \text{demand\_aware\_max\_min\_fair}(G, \mathcal{T}, \theta, \mathcal{R})$ 
3 return  $F$ 

```

Algorithm A.3: Demand-Aware Max-Min Fair.

Input: G . current network state.
Input: $\mathcal{T}_{1 \times n}$. source destination pairs (long flows).
Input: $\mathcal{R}_{1 \times n}$. the (sampled) routing for each flow.
Input: $\theta_{1 \times n}$. the drop-limited rate for each flow.
Output: $\hat{\theta}_{1 \times n}$. max-min fair rates after applying congestion.

```

1 for  $\forall f \in \mathcal{T}$  do
2    $e \leftarrow G.\text{add\_virtual\_edge}()$ 
3    $e.\text{capacity} = \theta_f$ 
4    $\mathcal{R}_f.\text{add}(e)$ 
5 end
6  $\hat{\theta} \leftarrow \text{network\_wide\_max\_min\_fairness}(G, \mathcal{T}, \mathcal{R})$ 
7 return  $\hat{\theta}$ 

```

network-wide max-min fair rates on this augmented topology. We can use the same method to enforce congestion control rate limits in the first few epochs of each flow in cases where the protocol's congestion window limits the flow's rate.

We can use any of the variants of the k -waterfilling algorithm [34] in the network-wide-max-min-fair function in Line 6 (but some can result in $O(|E| + |F|)$ number of iterations where $|E|$ is the number of edges and $|F|$ the number of flows). Instead, we use a faster variant of max-min fairness [45] that speeds up k -waterfilling by $30\times$ with minor degradation in the quality of the estimated rates (Fig. 11(c), Fig. 11(b)).

B Details of Offline Measurements

We measure three empirically driven distributions offline and use them to estimate CLPs in SWARM: (1) the upper bound on the throughput of long flows in a lossy network, (2) the necessary number of RTTs to deliver short flows, and (3) the queueing delay. In this section, we describe how we gathered these distributions. We also show a sample in Fig. A.8.

Throughput of long flows in a lossy network. We use Topol-

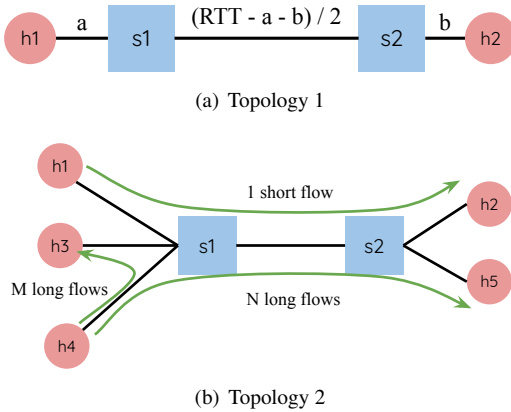


FIGURE A.1: Topologies for offline measurements.

ogy 1 in Fig. A.1 where $h1$ sends a long flow f to $h2$. We use `iperf3` to measure throughput under different network conditions (*e.g.*, by inducing packet drops in $s1 - s2$ or changing the RTT by adjusting the delay on $s1 - s2$). We ensure that link capacities are sufficiently high so that the drop rate always dictates the rate. We also repeat each experiment multiple times to create a statistically significant distribution [18].

Number of RTTs for short flows. We send a short flow from $h1$ to $h2$ in Topology 1 (Fig. A.1) under different settings (*e.g.*, flow size, slow start threshold, initial congestion window) and network conditions (*e.g.*, packet drop rate, RTT). We repeat each experiment multiple times, measure the FCT of the flow in each experiment, and construct an FCT distribution. We ignore the queueing delay since we only have one flow in this experiment and divide the FCT distribution by the two-way propagation delay to derive the necessary number of RTTs.

Queueing delay for short flows. Queueing delay is harder to measure as it depends on network load and the number of competing flows. We use Topology 2 in Fig. A.1 to estimate the queueing delay distribution under various network conditions. We route M long flows from $h4$ to $h3$ and N additional long flows from $h4$ to $h5$. After these long flows reach the steady state, we route a flow small enough to finish within an RTT from $h1$ to $h2$ and measure its FCT.

In this experiment, we can control the number of flows that compete on the $s1 - s2$ link using N and the utilization of the $s1 - s2$ link using M and N . The latter is because the $h4 - s1$ is the main bottleneck in the network and determines the rate of the N long flows that go through $s1 - s2$. We vary both M and N to model different network conditions and measure the FCT of the short flow. We subtract the two-way propagation delay from the FCT to derive the queueing delay. Once again, we repeat these experiments to ensure statistical significance in the resulting distributions.

C Additional Experiment Details

In this section, we describe the details of how we evaluate SWARM using emulation, simulation, and testbed experiments

on a set of representative and common incidents.

C.1 Traffic Traces

We evaluate our approach using many traffic traces to ensure our results are reliable. Each trace includes several network flows, each of which has 4 pieces of information: source, destination, size, and start time. We ensure these traces are long enough to capture the impact of mitigation on a wide range of flows with various sizes. To prevent capturing the impact of an empty network, we discard the performance of flows that begin in an initial window of the trace. Note that the duration of this window and the trace itself depend on the network bandwidth and delay in each setup.

Flow sizes. We follow recent works [38, 65] and sample flow sizes from a well-known and widely used distribution from DCTCP [5].

Flow start time. We generate flow start times using a Poisson distribution [35, 38, 41, 61] with inter-arrival time derived from Azure to ensure the load on the network is reasonable (unless otherwise specified).

Source and destination. We follow [38] in picking the source and destinations.

C.2 Failures Types

We evaluate our approach on different types of failures (see Table 2). These failures (*e.g.*, packet drop on multiple links or capacity drop due to fiber cut) are common in Azure and other cloud providers, as several studies confirm [7, 24, 31, 55, 63, 71].

We use Mininet (a network emulator) as our primary evaluation framework for the reasons outlined below. We provide specific details on the failure cases in Table A.1. These 57 cases represent a wide range of potential issues that could arise in a datacenter. For instance, we evaluate two scenarios per packet drop rate for single-link failure incidents, which cover all possible single-link failures that can occur in a datacenter. This is because (a) the target link is either between ToR and an aggregation or between an aggregation and a core switch [3], and (b) Clos is a symmetric topology. Therefore, all ToR-aggregation and core-aggregation links are equivalent [4]. This means it is enough to only look at two cases, which are representative of a wide range of possible incidents.

The same logic applies to other cases. For example, one of our two-link failure scenarios considers the case where two ToR-aggregation links in the same pod drop packets. This scenario represents any two ToR-aggregation links that may fail in any of the pods over the entire datacenter.

C.3 Emulation, Simulation, and Testbed

Emulation Setup (Mininet). Mininet uses a real, production-grade TCP/IP stack from the Linux kernel. This allows us to evaluate our methods on production-grade implementations of congestion control algorithms such as BBR [12] or Cubic [28]. However, scaling Mininet is challenging [52]. We use the Clos

		Details	#scenarios
Scen. 1	1 link failure	One T0 – T1 and one T1 – T2, each with two levels of packet drop rate $\sim 5\%$ and 0.005%	4
	2 link failures	Four combination of pair of links (two T0 – T1s in the same cluster connected to the same T0, two T0 – T1s in the same cluster connected to different T0s & T1s, one T0 – T1 & one T1 – T2 connected to different T1s, and two T1 – T2s connected to different T1s & T2s), each with all combinations of the two packet drop levels and all the possible ordering of failures.	32
Scen. 2	no other link failure	one T1 – T2 with capacity reduced to half	1
	1 other link failure	one T1 – T2 with capacity reduced to half, another T0 – T1 link with 3 levels of failure (two packet drop levels + completely down), and all possible ordering of failures.	6
Scen. 3	no link failure	One T0 with two levels of packet drop rate $\sim 5\%$ and 0.005%	2
	1 link failure	One T0 and One T0 – T1 in the same cluster connected to a different T0, with all combinations of packet drop rates (ToR with two levels of packet drop rate and link with three levels of failure (two packet drop rates + completely down)), and all the possible ordering of failures.	12
		total number of evaluated scenarios	57

TABLE A.1: Mininet Experiment Details.

topology from Fig. 2 with 8 servers, 4 ToRs, 4 aggregation switches (T1), and 4 spine switches (T2). Even with this setup, we required a server with 64 cores and 256 GB memory to achieve the desired load level. We include results from *4000 hours experiments (over 5 months)* to cover all 57 scenarios, all combinations of possible mitigations, and running each multiple times.

Emulating a network with gigabit-level bandwidth (40 Gbps) and microsecond-level propagation delay ($50 \mu s$) is impossible in Mininet. Therefore, we downscale the traffic and the link properties following [48, 50]. This involves (1) reducing the link capacities and increasing the link delay to ensure the network bandwidth-delay product remains the same and (2) increasing the inter-arrival time between flows by the same factor (in our case, $120\times$). Our testbed and simulation experiments compensate for this.

We tested our approach with both Cubic [28] and BBR [12] congestion control algorithms in Mininet (and DCTCP [5] in our NS3 simulations). Our findings suggest that the choice of mitigation depends less on the congestion control, and a common abstraction that ensures these flows receive max-min fair rates is enough (TCP objective [20]).

For our evaluation, we use traffic traces that last 500 seconds (around 9 minutes) to capture enough samples from different flow sizes. We only measure the impact on flows that start within [50, 150] seconds to avoid including the start-up phase and to ensure all the captured flows finish before the end of the traffic trace. We repeat each experiment for 30 different traffic traces (§C.1).

Simulation Setup (NS3). We use a Clos topology with 16 cores (T2), 32 aggregations (T1), 32 ToRs, and 128 servers. Each link has 20 Gbps bandwidth and $100 \mu s$ propagation delay. We use DCTCP [5] as the congestion control algorithm.

We consider an example where two links drop packets (one at a high rate $\sim 0.5\%$ and the other one at a low rate $\sim 0.005\%$). This scenario captures the impact of both the drop rates and the network load on the decision. We also find the packet drop rate impacts the scalability of the simulation. Thus, we had

to slightly reduce it compared to our Mininet experiments. Even with this drop rate, it takes NS3 one day to compute the performance of one mitigation on a single traffic trace. We use MPI to speed up NS3 and reduce the runtime.

Each of our traffic traces is 10 s long, and we measure the impact on flows that start within [0.5, 1) s. We repeat each experiment for 30 different traffic traces.

Testbed Setup. Our testbed has 32 servers connected using a Clos topology with six ToRs, four aggregation switches (T1), and two core switches (T2). Each link in our testbed has 10 Gbps bandwidth and $200 \mu s$ propagation delay. The topology is a Clos [3] where all T1 and T2 switches are connected to each other (different from Mininet and NS3 simulations).

Due to hardware limitations, we can only inject packet drops at the power two. We consider an example where we have two links dropping packets (one at a high rate $\frac{1}{16}$ and the other one at a low rate $\frac{1}{256}$).

C.4 Other Details

SWARM. We use 32 traffic traces and 1000 routing samples in SWARM (unless mentioned otherwise). For our baseline comparisons in Mininet, we set the duration of each traffic trace to 200 s, and as in Mininet, we compute the CLP metrics over all the flows that start within [50, 150] s. We also use 200 ms as the epoch size for Alg. 1. Note that an ideal epoch size should be in the order of the flow inter-arrival time (1 ms in Mininet). However, we find that SWARM can still find good mitigations even when we use a much larger epoch size.

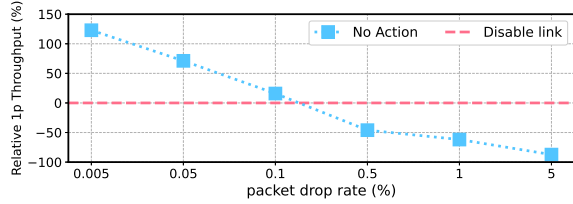
Routing. We assume either ECMP or WCMP routing [27, 70], which are typical in data centers.

D Extended Evaluation

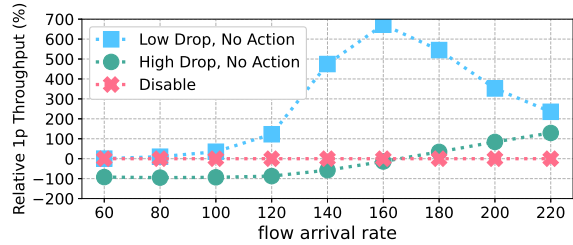
In this section, we present an extended evaluation of SWARM.

D.1 Sensitivity to Inputs

Packet drop rates. We compare the relative difference between the 1p throughput of taking no action and disabling a



(a) Sensitivity to packet drop rate



(b) Sensitivity to flow arrival rate changes

FIGURE A.2: Evaluating sensitivity of SWARM to errors in the input packet drop rate and flow arrival rate using Mininet. All experiments are in a scenario where a T0 – T1 drop packets.

link in a failure scenario where a T0 – T1 link drops packets at different rates (Fig. A.2). We find the choice of the right decision to be bi-modal with a wide room for error. It is better to take no action for all the drop rates below $\sim 0.1\%$ while the best action is to disable the link beyond that point. Also, the penalty close to this transition point (0.1% drop rate) is rather small. In other words, the error in the input packet loss rate has to be an order of magnitude for SWARM to make the wrong decision, an unlikely possibility in today’s clouds.

Flow arrival rates. We investigate how SWARM’s decisions change under two failure severities and different flow arrival rates (Fig. A.2). We observe that the gap between the two decisions is significant outside of a few inflection points, which means SWARM will pick the best mitigation in most cases.

For instance, in the high drop rate scenario, disabling the link is better than taking no action for arrival rates ranging from 60 to 160 flows per second (fps). Taking no action is better once we pass 160 fps as bringing down the link causes network congestion. This means SWARM has a wide margin of error. At small and large arrival rates, the difference between the two actions is consistently large, which means SWARM would pick the right action. At medium arrival rates, the difference is very small, so whichever action SWARM picks results in almost the same impact.

D.2 Sensitivity to Congestion Control

We conduct a limited experiment to evaluate whether SWARM is resilient to the choice of congestion control. We consider a scenario where a T0 – T1 and a T1 – T2 are dropping packets at low and high drop rates, respectively. We use two example congestion control protocols (CUBIC [28] and BBR [12]), which behave differently under loss. Cubic significantly re-

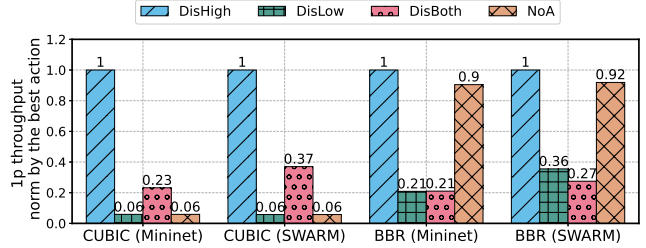


FIGURE A.3: Evaluation of SWARM on multiple congestion controls in Mininet. In general, SWARM is able to correctly order the mitigation actions and approximately capture the relative difference between different mitigation actions on both congestion controls (BBR [12] and Cubic [28]).

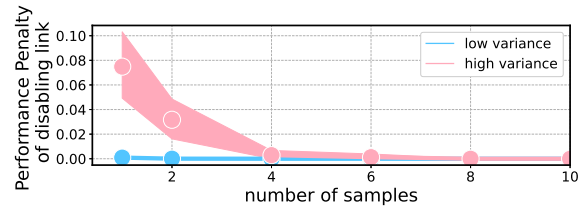
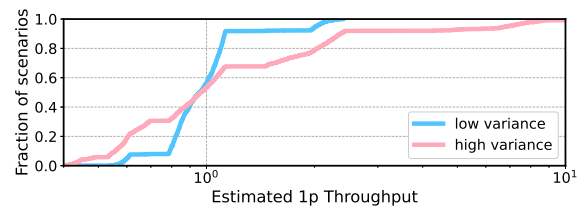


FIGURE A.4: Variance in the input flow arrival rate and its impact on the performance penalty of mitigation actions in Mininet.

duces its rate under loss while BBR does not.

We compare SWARM’s estimated 1p throughput with Mininet under four mitigations (Fig. A.3). We find (1) SWARM can estimate throughput adequately enough to pick the best mitigation and (2) the choice of the best action remains independent of the congestion control protocol. SWARM’s estimates are more accurate when it operates on the same congestion control protocol. This means it is more robust when operators correctly input the likelihood of different protocols being used inside their datacenters.

D.3 Assumptions and Design Choices

We also validate our assumptions and design choices.

Capturing the impact of drop-limited flows. We account for drop-limited flows by computing an upper bound on their rate and then enforce these bounds during max-min fair rate computations. We evaluate this assumption in an experiment where a single link delivers a varying number of flows and drops packets at different rates (Fig. A.5). We observe our observation is valid: each flow takes the minimum of its fair share rate and its drop-limited throughput.

We next evaluate our design choices for estimating the

distribution of the flow completion time and throughput (Fig. A.5). Specifically, we show the quality of SWARM’s approximations compared to Mininet measurements. We use a scenario where two links drop packets at different rates, and the mitigation is to disable the link with the higher drop rate.

Single Epoch vs. Multiple Epochs. In SWARM, we use epochs to capture the shift in network bottlenecks as flows arrive or depart over time. We observe that not capturing these dynamics leads to more than 50% estimation error on average (this is equivalent to running SWARM with only one epoch). Instead, SWARM uses multiple epochs to model flow arrival/departure and avoids this error.

Distributions vs deterministic estimates. In SWARM, we pick the best mitigation based on CLP distributions. This method increases confidence in the final decision and has less error compared to relying on a single CLP sample. SWARM can efficiently compute these distributions in parallel.

Queueing Delay. We show the importance of accounting for queuing delay using an example scenario where the link C0 – B0 in the Fig. 2 topology drops packets at a high drop rate and the operator prioritizes 99p FCT of short flows. In this case, the best action is to disable the link. After we apply the mitigation, the link C0 – B1 starts to drop packets at a high drop rate. Disabling C0 – B1 is not feasible anymore as it partitions the network, so we have to either take no action or bring back C0 – B0. If we ignore the queuing delay, both of these options have the same approximate 99p FCT. However, bringing back C0 – B0 is a better mitigation since it increases the path diversity and reduces the queuing delay (Table A.5).

D.4 Results for Other Comparators

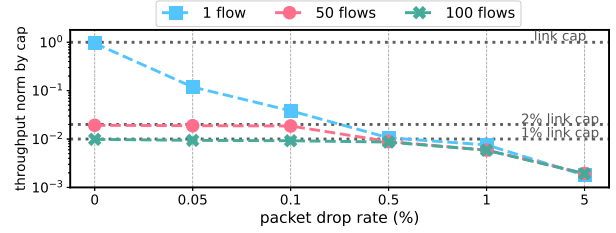
Priority1pT. This comparator minimizes the 1p (1st percentile) throughput. It uses two tiebreakers in the following order: average throughput and 99p FCT.

Linear combination. This comparator minimizes a weighted combination of the three CLP metrics (99p FCT, 1p throughput, and average throughput);

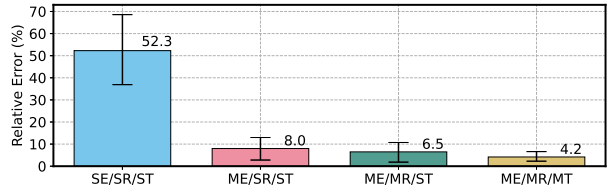
$$w_0 \frac{99p\ FCT}{99p\ FCT_h} + w_1 \frac{1p\ Thru_h}{1p\ Thru} + w_2 \frac{avg\ Thru_h}{avg\ Thru}$$

where w_i is the assigned weight and $Metric_h$ is the metric measured in a healthy network. Note that we prefer lower FCT but higher throughput. Therefore, we use their inverses in our definition of the linear comparator. SWARM admits any combination of weights, but we evaluate for the case where all weights are set to 1. This is different from all the other comparators as it does not set any preference for any metric.

Summary of the results. Fig. A.6 and Fig. A.7 compare the performance penalty of SWARM against other baselines across the three types of failures for Priority1pT and Linear combination comparator. In summary, SWARM’s performance penalty is low across all the scenarios and metrics.



(a) Drop-limited vs Capacity-limited



(b) Impact of different design choices on throughput estimation (E=Epoch, R=Routing Sample, T=Traffic Sample, S=Single, M=Multiple). For example, SE/SR/ST refers to using a single epoch, with a single routing sample, and a single traffic sample. We run each method with 10 different seeds.

Approach	Best Action	FCT Penalty
Ignore Queueing	Disable C0 – B0	48 %
Model Queueing	Bring back C0 – B0	0 %

(c) How queueing delay influences the mitigation choice.

FIGURE A.5: Validationg assumptions and design choices using Mininet. (a) shows flows are capacity- or loss-limited. A flow is loss-limited when its rate drops below its fair share of the link capacity (marked as dashed lines) (b) shows the impact of our design choices and the relative error with respect to Mininet. (c) the importance of accounting for queuing delay.

E Disabling the Congested Device

The routing protocols such as ECMP ignore the asymmetry in the datacenters, which can cause congestion. We can mitigate this by disabling the congested link or device so the routing can utilize other paths. For instance, each logical link between two switches [63] consists of multiple physical links. A control protocol multiplexes the packets over these physical links by hashing the packet’s header. When a cut happens in any of these physical links, the link capacity decreases and can cause congestion on the remaining physical links. To mitigate this, we can disable the logical link to enable ECMP/WCMP to use other paths with healthy links.

F SWARM’s Benefits

The reason behind SWARM’s benefit depends on the scenario. In some incidents, SWARM chooses the actions that are already supported but ignored by prior work. In some other cases, the benefit is due to the larger action space of SWARM. We show this using two example failures on the topology in Fig. 2.

Scenario 1 (same action space). In this case, a link between a tier-0 and tier-1 switch start dropping packets (e.g., C0 – B1). Existing threshold-based methods [63, 71] will take no

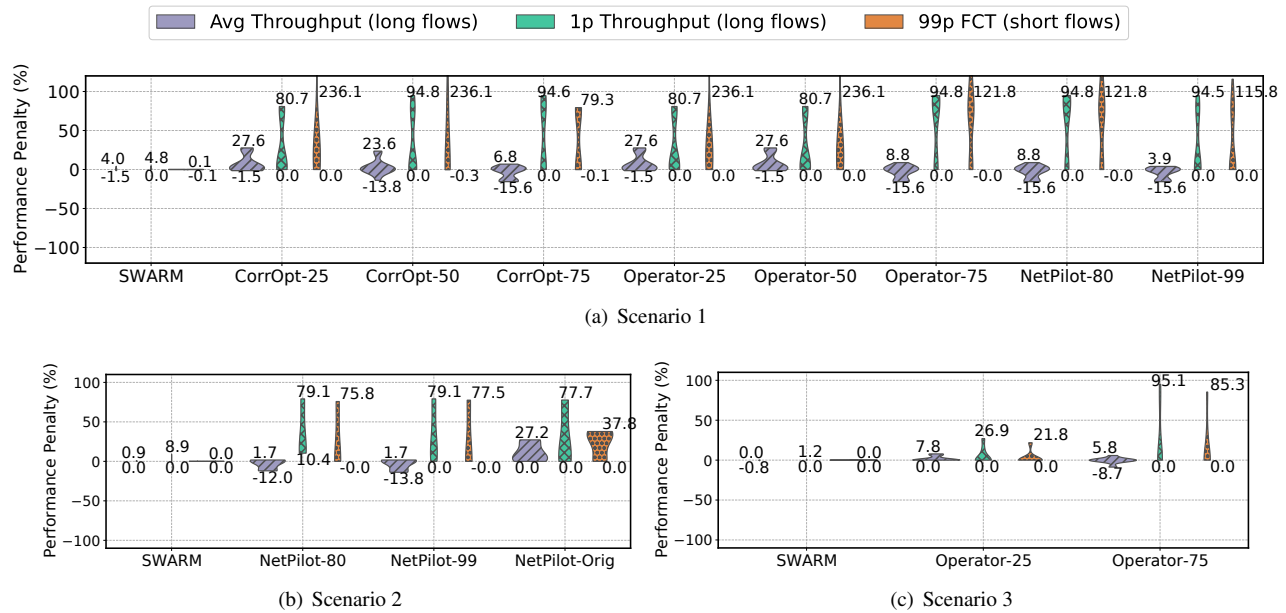
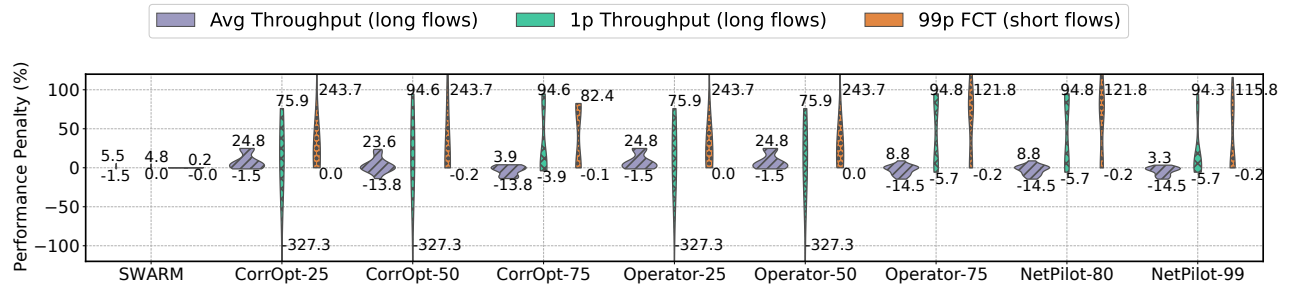


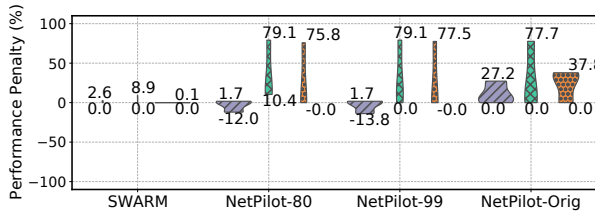
FIGURE A.6: SWARM outperforms other baselines under Priority1pT comparator in Mininet. It is the only method with a low penalty across all the metrics and all the scenarios.

action if they use a large threshold. This decision process completely ignores the impact of the packet drop rate. If the packet drop rate is high, we should disable the link. If these threshold-based methods use a smaller threshold, they end up always disabling the link. This decision completely ignores the impact of traffic load and the packet drop rate. Therefore, it can cause severe congestion. SWARM takes all these factors into account and finds much more effective mitigations (even though the action space is the same).

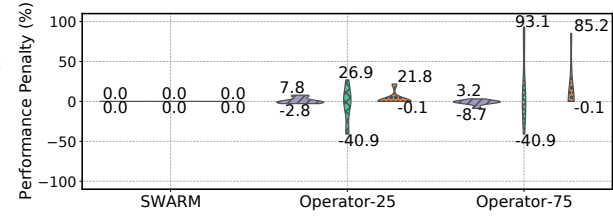
Scenario 2 (larger action space). Imagine a sequence of failures. First, a link between a tier-0 and tier-1 switch starts dropping packets at a moderate rate. SWARM would decide to disable the link. Later, another link between the same tier-0 switch and another tier-1 switch starts dropping packets at a much higher rate. In this case, SWARM disables the new link and re-enables the old link (undoing its previous action). This is because disabling both links would reduce the capacity of the network substantially, causing severe congestion. SWARM can reason about these cases, which enables exploring a broader set of actions, such as bringing back less faulty links.



(a) Scenario 1



(b) Scenario 2



(c) Scenario 3

FIGURE A.7: SWARM outperforms other baselines under Linear combination comparator in Mininet. SWARM consistently achieves low penalty (always $\leq 8.9\%$) across all the metrics and all the scenarios.

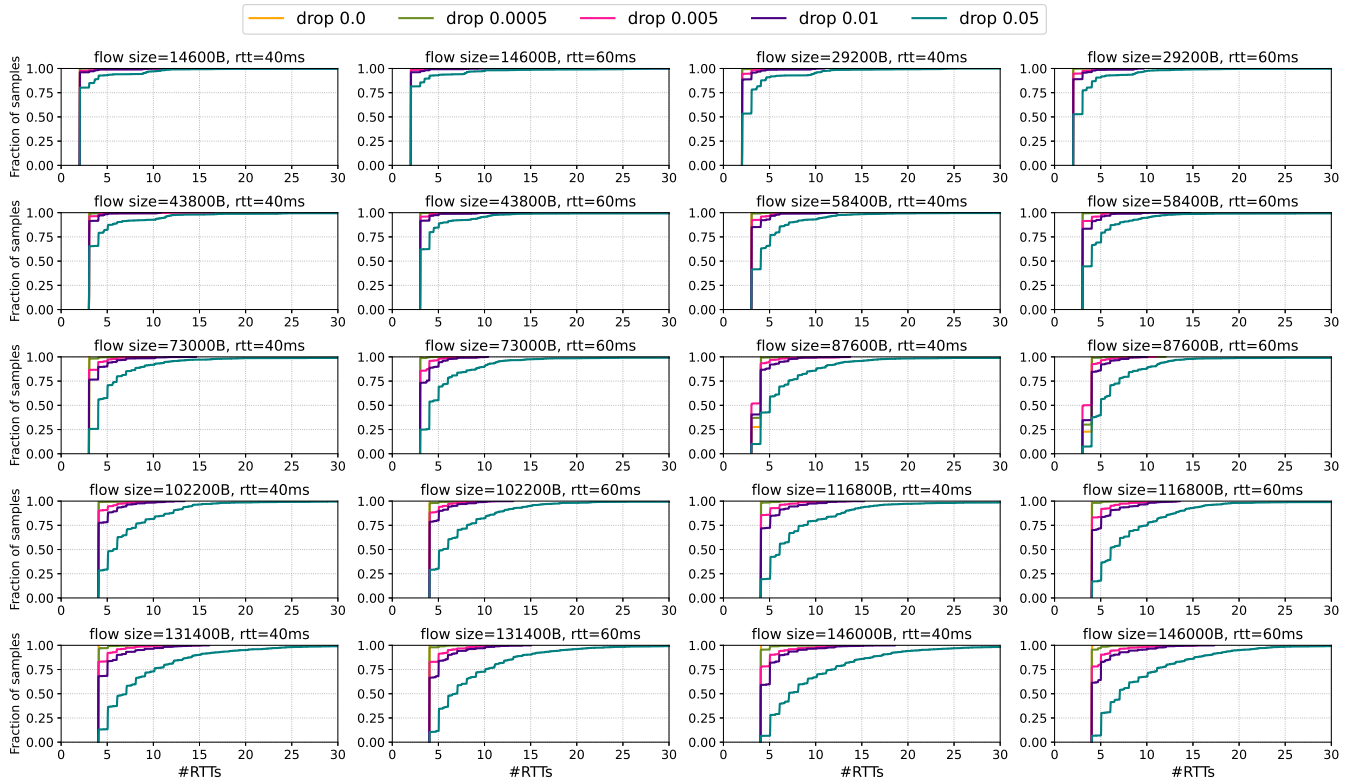


FIGURE A.8: An example of distribution measured for short flow's FCT using Mininet.